

Table of Contents

1. String Hashing
2. Rabin-Karp for String Matching
3. Prefix function - Knuth-Morris-Pratt
4. Z-function
5. Suffix Array
6. Aho-Corasick algorithm
7. Suffix Tree
8. Suffix Automaton
9. Lyndon factorization
10. Expression parsing
11. Manacher's Algorithm - Finding all sub-palindromes in $O(N)$
12. Finding repetitions

Translated

From: e-maxx.ru

String Hashing

Hashing algorithms are helpful in solving a lot of problems.

We want to solve the problem of comparing strings efficiently. The brute force way of doing so is just to compare the letters of both strings, which has a time complexity of $O(\min(n_1, n_2))$ if n_1 and n_2 are the sizes of the two strings. We want to do better. The idea behind the string hashing is the following: we map each string into an integer and compare those instead of the strings. Doing this allows us to reduce the execution time of the string comparison to $O(1)$.

For the conversion, we need a so-called **hash function**. The goal of it is to convert a string into an integer, the so-called **hash** of the string. The following condition has to hold: if two strings s and t are equal ($s = t$), then their hashes also have to be equal ($\text{hash}(s) = \text{hash}(t)$). Otherwise, we will not be able to compare strings.

Notice, the opposite direction doesn't have to hold. If the hashes are equal ($\text{hash}(s) = \text{hash}(t)$), then the strings do not necessarily have to be equal. E.g. a valid hash function would be simply $\text{hash}(s) = 0$ for each s . Now, this is just a stupid example, because this function will be completely useless, but it is a valid hash function. The reason why the opposite direction doesn't have to hold, is because there are exponentially many strings. If we only want this hash function to distinguish between all strings consisting of lowercase characters of length smaller than 15, then already the hash wouldn't fit into a 64-bit integer (e.g. unsigned long long) any more, because there are so many of them. And of course, we don't want to compare arbitrary long integers, because this will also have the complexity $O(n)$.

So usually we want the hash function to map strings onto numbers of a fixed range $[0, m)$, then comparing strings is just a comparison of two integers with a fixed length. And of course, we want $\text{hash}(s) \neq \text{hash}(t)$ to be very likely if $s \neq t$.

That's the important part that you have to keep in mind. Using hashing will not be 100% deterministically correct, because two complete different strings might have the same hash (the hashes collide). However, in a wide majority of tasks, this can be safely ignored as the probability of the hashes of two different strings colliding is still very small. And we will discuss some techniques in this article how to keep the probability of collisions very low.

Calculation of the hash of a string

The good and widely used way to define the hash of a string s of length n is

$$\begin{aligned}\text{hash}(s) &= s[0] + s[1] \cdot p + s[2] \cdot p^2 + \dots + s[n-1] \cdot p^{n-1} \mod m \\ &= \sum_{i=0}^{n-1} s[i] \cdot p^i \mod m,\end{aligned}$$

where p and m are some chosen, positive numbers. It is called a **polynomial rolling hash function**.

It is reasonable to make p a prime number roughly equal to the number of characters in the input alphabet. For example, if the input is composed of only lowercase letters of the English alphabet, $p = 31$ is a good choice. If the input may contain both uppercase and lowercase letters, then $p = 53$ is a possible choice. The code in this article will use $p = 31$.

Obviously m should be a large number since the probability of two random strings colliding is about $\approx \frac{1}{m}$. Sometimes $m = 2^{64}$ is chosen, since then the integer overflows of 64-bit integers work exactly like the modulo operation. However, there exists a method, which generates colliding strings (which work independently from the choice of p). So in practice, $m = 2^{64}$ is not recommended. A good choice for m is some large prime number. The code in this article will just use $m = 10^9 + 9$. This is a large number, but still small enough so that we can perform multiplication of two values using 64-bit integers.

Here is an example of calculating the hash of a string s , which contains only lowercase letters. We convert each character of s to an integer. Here we use the conversion $a \rightarrow 1, b \rightarrow 2, \dots, z \rightarrow 26$. Converting $a \rightarrow 0$ is not a good idea, because then the hashes of the strings $a, aa, aaa, \dots$ all evaluate to 0.

```

long long compute_hash(string const& s) {
    const int p = 31;
    const int m = 1e9 + 9;
    long long hash_value = 0;
    long long p_pow = 1;
    for (char c : s) {
        hash_value = (hash_value + (c - 'a' + 1) * p_pow) % m;
        p_pow = (p_pow * p) % m;
    }
    return hash_value;
}

```

Precomputing the powers of p might give a performance boost.

Example tasks

Search for duplicate strings in an array of strings

Problem: Given a list of n strings s_i , each no longer than m characters, find all the duplicate strings and divide them into groups.

From the obvious algorithm involving sorting the strings, we would get a time complexity of $O(nm \log n)$ where the sorting requires $O(n \log n)$ comparisons and each comparison take $O(m)$ time. However, by using hashes, we reduce the comparison time to $O(1)$, giving us an algorithm that runs in $O(nm + n \log n)$ time.

We calculate the hash for each string, sort the hashes together with the indices, and then group the indices by identical hashes.

```

vector<vector<int>> group_identical_strings(vector<string> const& s) {
    int n = s.size();
    vector<pair<long long, int>> hashes(n);
    for (int i = 0; i < n; i++)
        hashes[i] = {compute_hash(s[i]), i};

    sort(hashes.begin(), hashes.end());

    vector<vector<int>> groups;
    for (int i = 0; i < n; i++) {
        if (i == 0 || hashes[i].first != hashes[i-1].first)
            groups.emplace_back();
        groups.back().push_back(hashes[i].second);
    }
    return groups;
}

```

Fast hash calculation of substrings of given string

Problem: Given a string s and indices i and j , find the hash of the substring $s[i \dots j]$.

By definition, we have:

$$\text{hash}(s[i \dots j]) = \sum_{k=i}^j s[k] \cdot p^{k-i} \mod m$$

Multiplying by p^i gives:

$$\begin{aligned} \text{hash}(s[i \dots j]) \cdot p^i &= \sum_{k=i}^j s[k] \cdot p^k \mod m \\ &= \text{hash}(s[0 \dots j]) - \text{hash}(s[0 \dots i-1]) \mod m \end{aligned}$$

So by knowing the hash value of each prefix of the string s , we can compute the hash of any substring directly using this formula. The only problem that we face in calculating it is that we must be able to divide $\text{hash}(s[0 \dots j]) - \text{hash}(s[0 \dots i-1])$ by p^i . Therefore we need to

find the [modular multiplicative inverse](#) of p^i and then perform multiplication with this inverse. We can precompute the inverse of every p^i , which allows computing the hash of any substring of s in $O(1)$ time.

However, there does exist an easier way. In most cases, rather than calculating the hashes of substring exactly, it is enough to compute the hash multiplied by some power of p . Suppose we have two hashes of two substrings, one multiplied by p^i and the other by p^j . If $i < j$ then we multiply the first hash by p^{j-i} , otherwise, we multiply the second hash by p^{i-j} . By doing this, we get both the hashes multiplied by the same power of p (which is the maximum of i and j) and now these hashes can be compared easily with no need for any division.

Applications of Hashing

Here are some typical applications of Hashing:

- [Rabin-Karp algorithm](#) for pattern matching in a string in $O(n)$ time
- Calculating the number of different substrings of a string in $O(n^2)$ (see below)
- Calculating the number of palindromic substrings in a string.

Determine the number of different substrings in a string

Problem: Given a string s of length n , consisting only of lowercase English letters, find the number of different substrings in this string.

To solve this problem, we iterate over all substring lengths $l = 1 \dots n$. For every substring length l we construct an array of hashes of all substrings of length l multiplied by the same power of p . The number of different elements in the array is equal to the number of distinct substrings of length l in the string. This number is added to the final answer.

For convenience, we will use $h[i]$ as the hash of the prefix with i characters, and define $h[0] = 0$.

```
int count_unique_substrings(string const& s) {
    int n = s.size();

    const int p = 31;
    const int m = 1e9 + 9;
    vector<long long> p_pow(n);
    p_pow[0] = 1;
    for (int i = 1; i < n; i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(n + 1, 0);
    for (int i = 0; i < n; i++)
        h[i+1] = (h[i] + (s[i] - 'a' + 1) * p_pow[i]) % m;

    int cnt = 0;
    for (int l = 1; l <= n; l++) {
        unordered_set<long long> hs;
        for (int i = 0; i <= n - l; i++) {
            long long cur_h = (h[i + l] + m - h[i]) % m;
            cur_h = (cur_h * p_pow[n-i-1]) % m;
            hs.insert(cur_h);
        }
        cnt += hs.size();
    }
    return cnt;
}
```

Notice, that $O(n^2)$ is not the best possible time complexity for this problem. A solution with $O(n \log n)$ is described in the article about [Suffix Arrays](#), and it's even possible to compute it in $O(n)$ using a [Suffix Tree](#) or a [Suffix Automaton](#).

Improve no-collision probability

Quite often the above mentioned polynomial hash is good enough, and no collisions will happen during tests. Remember, the probability that collision happens is only $\approx \frac{1}{m}$. For $m = 10^9 + 9$ the probability is $\approx 10^{-9}$ which is quite low. But notice, that we only did one comparison. What if we compared a string s with 10^6 different strings. The probability that at least one collision happens is now $\approx 10^{-3}$. And if we want to

compare 10^6 different strings with each other (e.g. by counting how many unique strings exists), then the probability of at least one collision happening is already ≈ 1 . It is pretty much guaranteed that this task will end with a collision and returns the wrong result.

There is a really easy trick to get better probabilities. We can just compute two different hashes for each string (by using two different p , and/or different m , and compare these pairs instead. If m is about 10^9 for each of the two hash functions than this is more or less equivalent as having one hash function with $m \approx 10^{18}$. When comparing 10^6 strings with each other, the probability that at least one collision happens is now reduced to $\approx 10^{-6}$.

Practice Problems

- [Good Substrings - Codeforces](#)
- [A Needle in the Haystack - SPOJ](#)
- [String Hashing - Kattis](#)
- [Double Profiles - Codeforces](#)
- [Password - Codeforces](#)
- [SUB_PROB - SPOJ](#)
- [INSQ15_A](#)
- [SPOJ - Ada and Spring Cleaning](#)
- [GYM - Text Editor](#)
- [12012 - Detection of Extraterrestrial](#)
- [Codeforces - Games on a CD](#)
- [UVA 11855 - Buzzwords](#)
- [Codeforces - Santa Claus and a Palindrome](#)
- [Codeforces - String Compression](#)
- [Codeforces - Palindromic Characteristics](#)
- [SPOJ - Test](#)
- [Codeforces - Palindrome Degree](#)
- [Codeforces - Deletion of Repeats](#)
- [HackerRank - Gift Boxes](#)

Contributors:

Jakob Kogler (54.82%) Param Singh (19.74%) Gaurav Singh (8.33%) Morass (5.7%) Oleksandr Kulkov (3.95%) Bhuvnesh Jain (1.32%)
 Mariia Mykhailova (1.32%) MD. Minhazul Islam (0.88%) Mayank Pratap Singh (0.88%) Michael Hayter (0.88%) Arnar Bjarni Arnarson (0.44%)
 Hanzala Sohrab (0.44%) Siddharth S (0.44%) Tahanima Chowdhury (0.44%) Vedant Borkar (0.44%)

Translated From: e-maxx.ru

Rabin-Karp Algorithm for string matching

This algorithm is based on the concept of hashing, so if you are not familiar with string hashing, refer to the [string hashing](#) article.

This algorithm was authored by Rabin and Karp in 1987.

Problem: Given two strings - a pattern s and a text t , determine if the pattern appears in the text and if it does, enumerate all its occurrences in $O(|s| + |t|)$ time.

Algorithm: Calculate the hash for the pattern s . Calculate hash values for all the prefixes of the text t . Now, we can compare a substring of length $|s|$ with s in constant time using the calculated hashes. So, compare each substring of length $|s|$ with the pattern. This will take a total of $O(|t|)$ time. Hence the final complexity of the algorithm is $O(|t| + |s|)$: $O(|s|)$ is required for calculating the hash of the pattern and $O(|t|)$ for comparing each substring of length $|s|$ with the pattern.

Implementation

```
vector<int> rabin_karp(string const& s, string const& t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<long long> p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < (int)p_pow.size(); i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i+1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
    long long h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        long long cur_h = (h[i+S] + m - h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}
```

Practice Problems

- [SPOJ - Pattern Find](#)
- [Codeforces - Good Substrings](#)
- [Codeforces - Palindromic characteristics](#)
- [Leetcode - Longest Duplicate Substring](#)

Contributors:

[Jakob Kogler](#) (48.21%) [Param Singh](#) (26.79%) [Oleksandr Kulkov](#) (10.71%) [Bhuvnesh Jain](#) (5.36%) [Le Bao Hiep](#) (5.36%) [Joaquin Garmendia Cabrera](#) (1.79%) [Raghvendra Mishra](#) (1.79%)

Translated From: e-maxx.ru

Prefix function. Knuth–Morris–Pratt algorithm

Prefix function definition

You are given a string s of length n . The **prefix function** for this string is defined as an array π of length n , where $\pi[i]$ is the length of the longest proper prefix of the substring $s[0 \dots i]$ which is also a suffix of this substring. A proper prefix of a string is a prefix that is not equal to the string itself. By definition, $\pi[0] = 0$.

Mathematically the definition of the prefix function can be written as follows:

$$\pi[i] = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

For example, prefix function of string "abcbabcd" is [0, 0, 0, 1, 2, 3, 0], and prefix function of string "aabaaaab" is [0, 1, 0, 1, 2, 2, 3].

Trivial Algorithm

An algorithm which follows the definition of prefix function exactly is the following:

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 0; i < n; i++)
        for (int k = 0; k <= i; k++)
            if (s.substr(0, k) == s.substr(i-k+1, k))
                pi[i] = k;
    return pi;
}
```

It is easy to see that its complexity is $O(n^3)$, which has room for improvement.

Efficient Algorithm

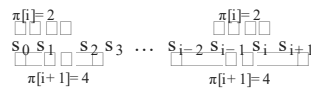
This algorithm was proposed by Knuth and Pratt and independently from them by Morris in 1977. It was used as the main function of a substring search algorithm.

First optimization

The first important observation is, that the values of the prefix function can only increase by at most one.

Indeed, otherwise, if $\pi[i+1] > \pi[i] + 1$, then we can take this suffix ending in position $i+1$ with the length $\pi[i+1]$ and remove the last character from it. We end up with a suffix ending in position i with the length $\pi[i+1] - 1$, which is better than $\pi[i]$, i.e. we get a contradiction.

The following illustration shows this contradiction. The longest proper suffix at position i that also is a prefix is of length 2, and at position $i+1$ it is of length 4. Therefore the string $s_0 s_1 s_2 s_3$ is equal to the string $s_{i-2} s_{i-1} s_i s_{i+1}$, which means that also the strings $s_0 s_1 s_2$ and $s_{i-2} s_{i-1} s_i$ are equal, therefore $\pi[i]$ has to be 3.



Thus when moving to the next position, the value of the prefix function can either increase by one, stay the same, or decrease by some amount.

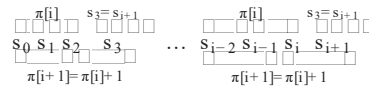
This fact already allows us to reduce the complexity of the algorithm to $O(n^2)$, because in one step the prefix function can grow at most by

one. In total the function can grow at most n steps, and therefore also only can decrease a total of n steps. This means we only have to perform $O(n)$ string comparisons, and reach the complexity $O(n^2)$.

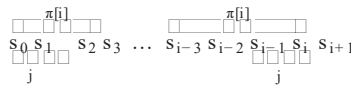
Second optimization

Let's go further, we want to get rid of the string comparisons. To accomplish this, we have to use all the information computed in the previous steps.

So let us compute the value of the prefix function π for $i + 1$. If $s[i + 1] = s[\pi[i]]$, then we can say with certainty that $\pi[i + 1] = \pi[i] + 1$, since we already know that the suffix at position i of length $\pi[i]$ is equal to the prefix of length $\pi[i]$. This is illustrated again with an example.

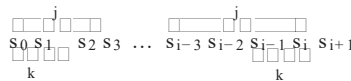


If this is not the case, $s[i + 1] \neq s[\pi[i]]$, then we need to try a shorter string. In order to speed things up, we would like to immediately move to the longest length $j < \pi[i]$, such that the prefix property in the position i holds, i.e. $s[0 \dots j - 1] = s[i - j + 1 \dots i]$:



Indeed, if we find such a length j , then we again only need to compare the characters $s[i + 1]$ and $s[j]$. If they are equal, then we can assign $\pi[i + 1] = j + 1$. Otherwise we will need to find the largest value smaller than j , for which the prefix property holds, and so on. It can happen that this goes until $j = 0$. If then $s[i + 1] = s[0]$, we assign $\pi[i + 1] = 1$, and $\pi[i + 1] = 0$ otherwise.

So we already have a general scheme of the algorithm. The only question left is how do we effectively find the lengths for j . Let's recap: for the current length j at the position i for which the prefix property holds, i.e. $s[0 \dots j - 1] = s[i - j + 1 \dots i]$, we want to find the greatest $k < j$, for which the prefix property holds.



The illustration shows, that this has to be the value of $\pi[j - 1]$, which we already calculated earlier.

Final algorithm

So we finally can build an algorithm that doesn't perform any string comparisons and only performs $O(n)$ actions.

Here is the final procedure:

- We compute the prefix values $\pi[i]$ in a loop by iterating from $i = 1$ to $i = n - 1$ ($\pi[0]$ just gets assigned with 0).
- To calculate the current value $\pi[i]$ we set the variable j denoting the length of the best suffix for $i - 1$. Initially $j = \pi[i - 1]$.
- Test if the suffix of length $j + 1$ is also a prefix by comparing $s[j]$ and $s[i]$. If they are equal then we assign $\pi[i] = j + 1$, otherwise we reduce j to $\pi[j - 1]$ and repeat this step.
- If we have reached the length $j = 0$ and still don't have a match, then we assign $\pi[i] = 0$ and go to the next index $i + 1$.

Implementation

The implementation ends up being surprisingly short and expressive.

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
```

```

while (j > 0 && s[i] != s[j])
    j = pi[j-1];
if (s[i] == s[j])
    j++;
pi[i] = j;
}
return pi;
}

```

This is an **online** algorithm, i.e. it processes the data as it arrives - for example, you can read the string characters one by one and process them immediately, finding the value of prefix function for each next character. The algorithm still requires storing the string itself and the previously calculated values of prefix function, but if we know beforehand the maximum value M the prefix function can take on the string, we can store only $M + 1$ first characters of the string and the same number of values of the prefix function.

Applications

Search for a substring in a string. The Knuth-Morris-Pratt algorithm

The task is the classical application of the prefix function.

Given a text t and a string s , we want to find and display the positions of all occurrences of the string s in the text t .

For convenience we denote with n the length of the string s and with m the length of the text t .

We generate the string $s + \# + t$, where $\#$ is a separator that appears neither in s nor in t . Let us calculate the prefix function for this string. Now think about the meaning of the values of the prefix function, except for the first $n + 1$ entries (which belong to the string s and the separator). By definition the value $\pi[i]$ shows the longest length of a substring ending in position i that coincides with the prefix. But in our case this is nothing more than the largest block that coincides with s and ends at position i . This length cannot be bigger than n due to the separator. But if equality $\pi[i] = n$ is achieved, then it means that the string s appears completely in at this position, i.e. it ends at position i . Just do not forget that the positions are indexed in the string $s + \# + t$.

Thus if at some position i we have $\pi[i] = n$, then at the position $i - (n + 1) - n + 1 = i - 2n$ in the string t the string s appears.

As already mentioned in the description of the prefix function computation, if we know that the prefix values never exceed a certain value, then we do not need to store the entire string and the entire function, but only its beginning. In our case this means that we only need to store the string $s + \#$ and the values of the prefix function for it. We can read one character at a time of the string t and calculate the current value of the prefix function.

Thus the Knuth-Morris-Pratt algorithm solves the problem in $O(n + m)$ time and $O(n)$ memory.

Counting the number of occurrences of each prefix

Here we discuss two problems at once. Given a string s of length n . In the first variation of the problem we want to count the number of appearances of each prefix $s[0 \dots i]$ in the same string. In the second variation of the problem another string t is given and we want to count the number of appearances of each prefix $s[0 \dots i]$ in t .

First we solve the first problem. Consider the value of the prefix function $\pi[i]$ at a position i . By definition it means that the prefix of length $\pi[i]$ of the string s occurs and ends at position i , and there is no longer prefix that follows this definition. At the same time shorter prefixes can end at this position. It is not difficult to see, that we have the same question that we already answered when we computed the prefix function itself: Given a prefix of length j that is a suffix ending at position i , what is the next smaller prefix $< j$ that is also a suffix ending at position i . Thus at the position i ends the prefix of length $\pi[i]$, the prefix of length $\pi[\pi[i] - 1]$, the prefix $\pi[\pi[\pi[i] - 1] - 1]$, and so on, until the index becomes zero. Thus we can compute the answer in the following way.

```

vector<int> ans(n + 1);
for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n-1; i > 0; i--)
    ans[pi[i-1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++;

```

Here for each value of the prefix function we first count how many times it occurs in the array π , and then compute the final answers: if we know that the length prefix i appears exactly $\text{ans}[i]$ times, then this number must be added to the number of occurrences of its longest suffix that is

also a prefix. At the end we need to add 1 to each result, since we also need to count the original prefixes also.

Now let us consider the second problem. We apply the trick from Knuth-Morris-Pratt: we create the string $s + \# + t$ and compute its prefix function. The only differences to the first task is, that we are only interested in the prefix values that relate to the string t , i.e. $\pi[i]$ for $i \geq n + 1$. With those values we can perform the exact same computations as in the first task.

The number of different substrings in a string

Given a string s of length n . We want to compute the number of different substrings appearing in it.

We will solve this problem iteratively. Namely we will learn, knowing the current number of different substrings, how to recompute this count by adding a character to the end.

So let k be the current number of different substrings in s , and we add the character c to the end of s . Obviously some new substrings ending in c will appear. We want to count these new substrings that didn't appear before.

We take the string $t = s + c$ and reverse it. Now the task is transformed into computing how many prefixes there are that don't appear anywhere else. If we compute the maximal value of the prefix function $\pi_{\max}$ of the reversed string t , then the longest prefix that appears in s is $\pi_{\max}$ long. Clearly also all prefixes of smaller length appear in it.

Therefore the number of new substrings appearing when we add a new character c is $|s| + 1 - \pi_{\max}$.

So for each character appended we can compute the number of new substrings in $O(n)$ times, which gives a time complexity of $O(n^2)$ in total.

It is worth noting, that we can also compute the number of different substrings by appending the characters at the beginning, or by deleting characters from the beginning or the end.

Compressing a string

Given a string s of length n . We want to find the shortest "compressed" representation of the string, i.e. we want to find a string t of smallest length such that s can be represented as a concatenation of one or more copies of t .

It is clear, that we only need to find the length of t . Knowing the length, the answer to the problem will be the prefix of s with this length.

Let us compute the prefix function for s . Using the last value of it we define the value $k = n - \pi[n - 1]$. We will show, that if k divides n , then k will be the answer, otherwise there is no effective compression and the answer is n .

Let n be divisible by k . Then the string can be partitioned into blocks of the length k . By definition of the prefix function, the prefix of length $n - k$ will be equal with its suffix. But this means that the last block is equal to the block before. And the block before has to be equal to the block before it. And so on. As a result, it turns out that all blocks are equal, therefore we can compress the string s to length k .

Of course we still need to show that this is actually the optimum. Indeed, if there was a smaller compression than k , than the prefix function at the end would be greater than $n - k$. Therefore k is really the answer.

Now let us assume that n is not divisible by k . We show that this implies that the length of the answer is n . We prove it by contradiction. Assuming there exists an answer, and the compression has length p (p divides n). Then the last value of the prefix function has to be greater than $n - p$, i.e. the suffix will partially cover the first block. Now consider the second block of the string. Since the prefix is equal with the suffix, and both the prefix and the suffix cover this block and their displacement relative to each other k does not divide the block length p (otherwise k divides n), then all the characters of the block have to be identical. But then the string consists of only one character repeated over and over, hence we can compress it to a string of size 1, which gives $k = 1$, and k divides n . Contradiction.

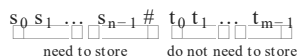
$s_0 \ s_1 \ s_2 \ s_3 \ s_4 \ s_5 \ s_6 \ s_7$

$s_0 \ s_1 \ s_2 \ s_3 \ s_4 \ s_5 \ s_6 \ s_7$
 $\pi[7] = 5$

$$s_4 = s_3, s_5 = s_4, s_6 = s_5, s_7 = s_6 \Rightarrow s_0 = s_1 = s_2 = s_3$$

Building an automaton according to the prefix function

Let's return to the concatenation of the two strings through a separator, i.e. for the strings s and t we compute the prefix function for the string $s + \# + t$. Obviously, since $\#$ is a separator, the value of the prefix function will never exceed $|s|$. It follows, that it is sufficient to only store the string $s + \#$ and the values of the prefix function for it, and we can compute the prefix function for all subsequent character on the fly:



Indeed, in such a situation, knowing the next character $c \in t$ and the value of the prefix function of the previous position is enough information to compute the next value of the prefix function, without using any previous characters of the string t and the value of the prefix function in them.

In other words, we can construct an **automaton** (a finite state machine): the state in it is the current value of the prefix function, and the transition from one state to another will be performed via the next character.

Thus, even without having the string t , we can construct such a transition table $(old_{\pi}, c) \rightarrow new_{\pi}$ using the same algorithm as for calculating the transition table:

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            int j = i;
            while (j > 0 && 'a' + c != s[j])
                j = pi[j-1];
            if ('a' + c == s[j])
                j++;
            aut[i][c] = j;
        }
    }
}
```

However in this form the algorithm runs in $O(n^2 26)$ time for the lowercase letters of the alphabet. Note that we can apply dynamic programming and use the already calculated parts of the table. Whenever we go from the value j to the value $\pi[j - 1]$, we actually mean that the transition (j, c) leads to the same state as the transition as $(\pi[j - 1], c)$, and this answer is already accurately computed.

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i-1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

As a result we construct the automaton in $O(26n)$ time.

When is such an automaton useful? To begin with, remember that we use the prefix function for the string $s + \# + t$ and its values mostly for a single purpose: find all occurrences of the string s in the string t .

Therefore the most obvious benefit of this automaton is the **acceleration of calculating the prefix function** for the string $s + \# + t$. By building the automaton for $s + \#$, we no longer need to store the string s or the values of the prefix function in it. All transitions are already computed in the table.

But there is a second, less obvious, application. We can use the automaton when the string t is a **gigantic string constructed using some rules**. This can for instance be the Gray strings, or a string formed by a recursive combination of several short strings from the input.

For completeness we will solve such a problem: given a number $k \leq 10^5$ and a string s of length $\leq 10^5$. We have to compute the number of occurrences of s in the k -th Gray string. Recall that Gray's strings are defined in the following way:

```
g1 = "a"
g2 = "aba"
g3 = "abacaba"
g4 = "abacabadabacaba"
```

In such cases even constructing the string t will be impossible, because of its astronomical length. The k -th Gray string is $2^k - 1$ characters long. However we can calculate the value of the prefix function at the end of the string effectively, by only knowing the value of the prefix function at the start.

In addition to the automaton itself, we also compute values $G[i][j]$ - the value of the automaton after processing the string g_i starting with the state j . And additionally we compute values $K[i][j]$ - the number of occurrences of s in g_i , before during the processing of g_i starting with the state j . Actually $K[i][j]$ is the number of times that the prefix function took the value $|s|$ while performing the operations. The answer to the problem will then be $K[k][0]$.

How can we compute these values? First the basic values are $G[0][j] = j$ and $K[0][j] = 0$. And all subsequent values can be calculated from the previous values and using the automaton. To calculate the value for some i we remember that the string g_i consists of g_{i-1} , the i character of the alphabet, and g_{i-1} . Thus the automaton will go into the state:

$$mid = \text{aut}[G[i-1][j]][i]$$

$$G[i][j] = G[i-1][mid]$$

The values for $K[i][j]$ can also be easily counted.

$$K[i][j] = K[i-1][j] + (mid == |s|) + K[i-1][mid]$$

So we can solve the problem for Gray strings, and similarly also a huge number of other similar problems. For example the exact same method also solves the following problem: we are given a string s and some patterns t_i , each of which is specified as follows: it is a string of ordinary characters, and there might be some recursive insertions of the previous strings of the form t_k^{cnt} , which means that at this place we have to insert the string t_k cnt times. An example of such patterns:

```
t1 = "abdeca"
t2 = "abc" + t130 + "abd"
t3 = t250 + t1100
t4 = t210 + t3100
```

The recursive substitutions blow the string up, so that their lengths can reach the order of 100^{100} .

We have to find the number of times the string s appears in each of the strings.

The problem can be solved in the same way by constructing the automaton of the prefix function, and then we calculate the transitions in for each pattern by using the previous results.

Practice Problems

- [UVA # 455 "Periodic Strings"](#)
- [UVA # 11022 "String Factoring"](#)
- [UVA # 11452 "Dancing the Cheeky-Cheeky"](#)
- [UVA 12604 - Caesar Cipher](#)
- [UVA 12467 - Secret Word](#)

- [UVA 11019 - Matrix Matcher](#)
- [SPOJ - Pattern Find](#)
- [SPOJ - A Needle in the Haystack](#)
- [Codeforces - Anthem of Berland](#)
- [Codeforces - MUH and Cube Walls](#)
- [Codeforces - Prefixes and Suffixes](#)

Contributors:

[Jakob Kogler](#) (80.62%) [Mariia Mykhailova](#) (14.21%) [Oleksandr Kulkov](#) (2.07%) [Morass](#) (1.29%) [Joaquin Garmendia Cabrera](#) (0.52%)
[Ashutosh Kumar Verma](#) (0.26%) [Bergur Snorrason](#) (0.26%) [Gaurang Tandon](#) (0.26%) [Harshit Jain](#) (0.26%) [Shaan Nobee](#) (0.26%)

Translated From: e-maxx.ru

Z-function and its calculation

Suppose we are given a string s of length n . The **Z-function** for this string is an array of length n where the i -th element is equal to the greatest number of characters starting from the position i that coincide with the first characters of s .

In other words, $z[i]$ is the length of the longest string that is, at the same time, a prefix of s and a prefix of the suffix of s starting at i .

Note. In this article, to avoid ambiguity, we assume 0-based indexes; that is: the first character of s has index 0 and the last one has index $n - 1$.

The first element of Z-function, $z[0]$, is generally not well defined. In this article we will assume it is zero (although it doesn't change anything in the algorithm implementation).

This article presents an algorithm for calculating the Z-function in $O(n)$ time, as well as various of its applications.

Examples

For example, here are the values of the Z-function computed for different strings:

- "aaaaa" - [0, 4, 3, 2, 1]
- "aabaab" - [0, 2, 1, 0, 2, 1, 0]
- "abacaba" - [0, 0, 1, 0, 3, 0, 1]

Trivial algorithm

Formal definition can be represented in the following elementary $O(n^2)$ implementation.

```
vector<int> z_function_trivial(string s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1; i < n; i++) {
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
    }
    return z;
}
```

We just iterate through every position i and update $z[i]$ for each one of them, starting from $z[i] = 0$ and incrementing it as long as we don't find a mismatch (and as long as we don't reach the end of the line).

Of course, this is not an efficient implementation. We will now show the construction of an efficient implementation.

Efficient algorithm to compute the Z-function

To obtain an efficient algorithm we will compute the values of $z[i]$ in turn from $i = 1$ to $n - 1$ but at the same time, when computing a new value, we'll try to make the best use possible of the previously computed values.

For the sake of brevity, let's call **segment matches** those substrings that coincide with a prefix of s . For example, the value of the desired Z-function $z[i]$ is the length of the segment match starting at position i (and that ends at position $i + z[i] - 1$).

To do this, we will keep the **[l, r) indices of the rightmost segment match**. That is, among all detected segments we will keep the one that ends rightmost. In a way, the index r can be seen as the "boundary" to which our string s has been scanned by the algorithm; everything beyond that point is not yet known.

Then, if the current index (for which we have to compute the next value of the Z-function) is i , we have one of two options:

- $i \geq r$ -- the current position is **outside** of what we have already processed.

We will then compute $z[i]$ with the **trivial algorithm** (that is, just comparing values one by one). Note that in the end, if $z[i] > 0$, we'll have to update the indices of the rightmost segment, because it's guaranteed that the new $r = i + z[i]$ is better than the previous r .

- $i < r$ -- the current position is inside the current segment match $[l, r)$.

Then we can use the already calculated Z-values to "initialize" the value of $z[i]$ to something (it sure is better than "starting from zero"), maybe even some big number.

For this, we observe that the substrings $s[l \dots r)$ and $s[0 \dots r - l)$ **match**. This means that as an initial approximation for $z[i]$ we can take the value already computed for the corresponding segment $s[0 \dots r - l)$, and that is $z[i - l]$.

However, the value $z[i - l]$ could be too large: when applied to position i it could exceed the index r . This is not allowed because we know nothing about the characters to the right of r : they may differ from those required.

Here is an **example** of a similar scenario:

`s = "aaaabaa"`

When we get to the last position ($i = 6$), the current match segment will be $[5, 7)$. Position 6 will then match position $6 - 5 = 1$, for which the value of the Z-function is $z[1] = 3$. Obviously, we cannot initialize $z[6]$ to 3, it would be completely incorrect. The maximum value we could initialize it to is 1 -- because it's the largest value that doesn't bring us beyond the index r of the match segment $[l, r)$.

Thus, as an **initial approximation** for $z[i]$ we can safely take:

$$z_0[i] = \min(r - i, z[i - l])$$

After having $z[i]$ initialized to $z_0[i]$, we try to increment $z[i]$ by running the **trivial algorithm** -- because in general, after the border r , we cannot know if the segment will continue to match or not.

Thus, the whole algorithm is split in two cases, which differ only in **the initial value** of $z[i]$: in the first case it's assumed to be zero, in the second case it is determined by the previously computed values (using the above formula). After that, both branches of this algorithm can be reduced to the implementation of **the trivial algorithm**, which starts immediately after we specify the initial value.

The algorithm turns out to be very simple. Despite the fact that on each iteration the trivial algorithm is run, we have made significant progress, having an algorithm that runs in linear time. Later on we will prove that the running time is linear.

Implementation

Implementation turns out to be rather concise:

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Comments on this implementation

The whole solution is given as a function which returns an array of length n -- the Z-function of s .

Array z is initially filled with zeros. The current rightmost match segment is assumed to be $[0; 0)$ (that is, a deliberately small segment which doesn't contain any i).

Inside the loop for $i = 1 \dots n - 1$ we first determine the initial value $z[i]$ – it will either remain zero or be computed using the above formula.

Thereafter, the trivial algorithm attempts to increase the value of $z[i]$ as much as possible.

In the end, if it's required (that is, if $i + z[i] > r$), we update the rightmost match segment $[l, r)$.

Asymptotic behavior of the algorithm

We will prove that the above algorithm has a running time that is linear in the length of the string – thus, it's $O(n)$.

The proof is very simple.

We are interested in the nested `while` loop, since everything else is just a bunch of constant operations which sums up to $O(n)$.

We will show that **each iteration** of the `while` loop will increase the right border r of the match segment.

To do that, we will consider both branches of the algorithm:

- $i \geq r$

In this case, either the `while` loop won't make any iteration (if $s[0] \neq s[i]$), or it will take a few iterations, starting at position i , each time moving one character to the right. After that, the right border r will necessarily be updated.

So we have found that, when $i \geq r$, each iteration of the `while` loop increases the value of the new r index.

- $i < r$

In this case, we initialize $z[i]$ to a certain value z_0 given by the above formula. Let's compare this initial value z_0 to the value $r - i$. We will have three cases:

- $z_0 < r - i$

We prove that in this case no iteration of the `while` loop will take place.

It's easy to prove, for example, by contradiction: if the `while` loop made at least one iteration, it would mean that initial approximation $z[i] = z_0$ was inaccurate (less than the match's actual length). But since $s[l \dots r)$ and $s[0 \dots r - 1)$ are the same, this would imply that $z[i - 1]$ holds the wrong value (less than it should be).

Thus, since $z[i - 1]$ is correct and it is less than $r - i$, it follows that this value coincides with the required value $z[i]$.

- $z_0 = r - i$

In this case, the `while` loop can make a few iterations, but each of them will lead to an increase in the value of the r index because we will start comparing from $s[r]$, which will climb beyond the $[l, r)$ interval.

- $z_0 > r - i$

This option is impossible, by definition of z_0 .

So, we have proved that each iteration of the inner loop make the r pointer advance to the right. Since r can't be more than $n - 1$, this means that the inner loop won't make more than $n - 1$ iterations.

As the rest of the algorithm obviously works in $O(n)$, we have proved that the whole algorithm for computing Z-functions runs in linear time.

Applications

We will now consider some uses of Z-functions for specific tasks.

These applications will be largely similar to applications of [prefix function](#).

Search the substring

To avoid confusion, we call t the **string of text**, and p the **pattern**. The problem is: find all occurrences of the pattern p inside the text t .

To solve this problem, we create a new string $s = p + \diamond + t$, that is, we apply string concatenation to p and t but we also put a separator character $\diamond$ in the middle (we'll choose $\diamond$ so that it will certainly not be present anywhere in the strings p or t).

Compute the Z-function for s . Then, for any i in the interval $[0; \text{length}(t) - 1]$, we will consider the corresponding value $k = z[i + \text{length}(p) + 1]$. If k is equal to $\text{length}(p)$ then we know there is one occurrence of p in the i -th position of t , otherwise there is no occurrence of p in the i -th position of t .

The running time (and memory consumption) is $O(\text{length}(t) + \text{length}(p))$.

Number of distinct substrings in a string

Given a string s of length n , count the number of distinct substrings of s .

We'll solve this problem iteratively. That is: knowing the current number of different substrings, recalculate this amount after adding to the end of s one character.

So, let k be the current number of distinct substrings of s . We append a new character c to s . Obviously, there can be some new substrings ending in this new character c (namely, all those strings that end with this symbol and that we haven't encountered yet).

Take a string $t = s + c$ and invert it (write its characters in reverse order). Our task is now to count how many prefixes of t are not found anywhere else in t . Let's compute the Z-function of t and find its maximum value $z_{\max}$. Obviously, t 's prefix of length $z_{\max}$ occurs also somewhere in the middle of t . Clearly, shorter prefixes also occur.

So, we have found that the number of new substrings that appear when symbol c is appended to s is equal to $\text{length}(t) - z_{\max}$.

Consequently, the running time of this solution is $O(n^2)$ for a string of length n .

It's worth noting that in exactly the same way we can recalculate, still in $O(n)$ time, the number of distinct substrings when appending a character in the beginning of the string, as well as when removing it (from the end or the beginning).

String compression

Given a string s of length n . Find its shortest "compressed" representation, that is: find a string t of shortest length such that s can be represented as a concatenation of one or more copies of t .

A solution is: compute the Z-function of s , loop through all i such that i divides n . Stop at the first i such that $i + z[i] = n$. Then, the string s can be compressed to the length i .

The proof for this fact is the same as the solution which uses the [prefix function](#).

Practice Problems

- [CSES - Finding Borders](#)
- [eolymp - Blocks of string](#)
- [Codeforces - Password \[Difficulty: Easy\]](#)
- [UVA # 455 "Periodic Strings" \[Difficulty: Medium\]](#)
- [UVA # 11022 "String Factoring" \[Difficulty: Medium\]](#)
- [UVa 11475 - Extend to Palindrome](#)
- [LA 6439 - Pasti Pas!](#)
- [Codechef - Chef and Strings](#)
- [Codeforces - Prefixes and Suffixes](#)

Contributors:

[William Di Luigi](#) (66.98%) [Oleksandr Kulkov](#) (20.0%) [Bhuvnesh Jain](#) (4.19%) [Mariia Mykhailova](#) (3.26%) [Anthony](#) (0.93%) [Jakob Kogler](#) (0.93%)
[huggin](#) (0.93%) [Eduardo Quirino](#) (0.47%) [Joaquin Garmendia Cabrera](#) (0.47%) [Madhur Chauhan](#) (0.47%) [Nafis](#) (0.47%) [PAVAN BANSAL](#) (0.47%)
[km6102](#) (0.47%)

Translated From: e-maxx.ru

Suffix Array

Definition

Let s be a string of length n . The i -th suffix of s is the substring $s[i \dots n - 1]$.

A **suffix array** will contain integers that represent the **starting indexes** of all the suffixes of a given string, after the aforementioned suffixes are sorted.

As an example look at the string $s = \text{abaa}b$. All suffixes are as follows

- 0. `abaa``b`
- 1. `baa``b`
- 2. `aa``b`
- 3. `a``b`
- 4. `b`

After sorting these strings:

- 2. `aa``b`
- 3. `a``b`
- 0. `abaa``b`
- 4. `b`
- 1. `baa``b`

Therefore the suffix array for s will be $(2, 3, 0, 4, 1)$.

As a data structure it is widely used in areas such as data compression, bioinformatics and, in general, in any area that deals with strings and string matching problems.

Construction

$O(n^2 \log n)$ approach

This is the most naive approach. Get all the suffixes and sort them using quicksort or mergesort and simultaneously retain their original indices. Sorting uses $O(n \log n)$ comparisons, and since comparing two strings will additionally take $O(n)$ time, we get the final complexity of $O(n^2 \log n)$.

$O(n \log n)$ approach

Strictly speaking the following algorithm will not sort the suffixes, but rather the cyclic shifts of a string. However we can very easily derive an algorithm for sorting suffixes from it: it is enough to append an arbitrary character to the end of the string which is smaller than any character from the string. It is common to use the symbol $\$$. Then the order of the sorted cyclic shifts is equivalent to the order of the sorted suffixes, as demonstrated here with the string `da``bbb`.

1. abbb\$d abbb
4. b\$dabb b
3. bb\$dab bb
2. bbb\$da bbb
0. dabbb\$ dabbb

Since we are going to sort cyclic shifts, we will consider **cyclic substrings**. We will use the notation $s[i \dots j]$ for the substring of s even if $i > j$. In this case we actually mean the string $s[i \dots n - 1] + s[0 \dots j]$. In addition we will take all indices modulo the length of s , and will omit the modulo operation for simplicity.

The algorithm we discuss will perform $\lceil \log n \rceil + 1$ iterations. In the k -th iteration ($k = 0 \dots \lceil \log n \rceil$) we sort the n cyclic substrings of s of length 2^k . After the $\lceil \log n \rceil$ -th iteration the substrings of length $2^{\lceil \log n \rceil} \geq n$ will be sorted, so this is equivalent to sorting the cyclic shifts altogether.

In each iteration of the algorithm, in addition to the permutation $p[0 \dots n - 1]$, where $p[i]$ is the index of the i -th substring (starting at i and with length 2^k) in the sorted order, we will also maintain an array $c[0 \dots n - 1]$, where $c[i]$ corresponds to the **equivalence class** to which the substring belongs. Because some of the substrings will be identical, and the algorithm needs to treat them equally. For convenience the classes will be labeled by numbers started from zero. In addition the numbers $c[i]$ will be assigned in such a way that they preserve information about the order: if one substring is smaller than the other, then it should also have a smaller class label. The number of equivalence classes will be stored in a variable `classes`.

Let's look at an example. Consider the string $s = \text{aaba}$. The cyclic substrings and the corresponding arrays $p[]$ and $c[]$ are given for each iteration:

0 :	(a, a, b, a)	$p = (0, 1, 3, 2)$	$c = (0, 0, 1, 0)$
1 :	(aa, ab, ba, aa)	$p = (0, 3, 1, 2)$	$c = (0, 1, 2, 0)$
2 :	(aaba, abaa, baaa, aaab)	$p = (3, 0, 1, 2)$	$c = (1, 2, 3, 0)$

It is worth noting that the values of $p[]$ can be different. For example in the 0-th iteration the array could also be $p = (3, 1, 0, 2)$ or $p = (3, 0, 1, 2)$. All these options permutation the substrings into a sorted order. So they are all valid. At the same time the array $c[]$ is fixed, there can be no ambiguities.

Let us now focus on the implementation of the algorithm. We will write a function that takes a string s and returns the permutations of the sorted cyclic shifts.

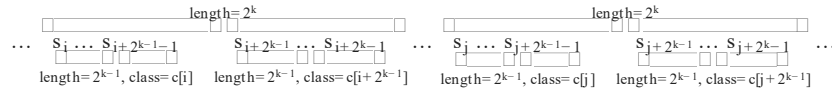
```
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;
```

At the beginning (in the **0-th iteration**) we must sort the cyclic substrings of length 1, that is we have to sort all characters of the string and divide them into equivalence classes (same symbols get assigned to the same class). This can be done trivially, for example, by using **counting sort**. For each character we count how many times it appears in the string, and then use this information to create the array $p[]$. After that we go through the array $p[]$ and construct $c[]$ by comparing adjacent characters.

```
vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
for (int i = 0; i < n; i++)
    cnt[s[i]]++;
for (int i = 1; i < alphabet; i++)
    cnt[i] += cnt[i-1];
for (int i = 0; i < n; i++)
    p[--cnt[s[i]]] = i;
c[p[0]] = 0;
int classes = 1;
for (int i = 1; i < n; i++) {
    if (s[p[i]] != s[p[i-1]])
        classes++;
    c[p[i]] = classes - 1;
}
```

Now we have to talk about the iteration step. Let's assume we have already performed the $k-1$ -th step and computed the values of the arrays $p[]$ and $c[]$ for it. We want to compute the values for the k -th step in $O(n)$ time. Since we perform this step $O(\log n)$ times, the complete algorithm will have a time complexity of $O(n \log n)$.

To do this, note that the cyclic substrings of length 2^k consists of two substrings of length 2^{k-1} which we can compare with each other in $O(1)$ using the information from the previous phase - the values of the equivalence classes $c[]$. Thus, for two substrings of length 2^k starting at position i and j , all necessary information to compare them is contained in the pairs $(c[i], c[i + 2^{k-1}])$ and $(c[j], c[j + 2^{k-1}])$.



This gives us a very simple solution: **sort** the substrings of length 2^k **by these pairs of numbers**. This will give us the required order $p[]$.

However a normal sort runs in $O(n \log n)$ time, with which we are not satisfied. This will only give us an algorithm for constructing a suffix array in $O(n \log^2 n)$ times.

How do we quickly perform such a sorting of the pairs? Since the elements of the pairs do not exceed n , we can use counting sort again. However sorting pairs with counting sort is not the most efficient. To achieve a better hidden constant in the complexity, we will use another trick.

We use here the technique on which **radix sort** is based: to sort the pairs we first sort them by the second element, and then by the first element (with a stable sort, i.e. sorting without breaking the relative order of equal elements). However the second elements were already sorted in the previous iteration. Thus, in order to sort the pairs by the second elements, we just need to subtract 2^{k-1} from the indices in $p[]$ (e.g. if the smallest substring of length 2^{k-1} starts at position i , then the substring of length 2^k with the smallest second half starts at $i - 2^{k-1}$).

So only by simple subtractions we can sort the second elements of the pairs in $p[]$. Now we need to perform a stable sort by the first elements. As already mentioned, this can be accomplished with counting sort.

The only thing left is to compute the equivalence classes $c[]$, but as before this can be done by simply iterating over the sorted permutation $p[]$ and comparing neighboring pairs.

Here is the remaining implementation. We use temporary arrays $pn[]$ and $cn[]$ to store the permutation by the second elements and the new equivalent class indices.

```
vector<int> pn(n), cn(n);
for (int h = 0; (1 << h) < n; ++h) {
    for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0)
            pn[i] += n;
    }
    fill(cnt.begin(), cnt.begin() + classes, 0);
    for (int i = 0; i < n; i++)
        cnt[c[pn[i]]]++;
    for (int i = 1; i < classes; i++)
        cnt[i] += cnt[i-1];
    for (int i = n-1; i >= 0; i--)
        p[--cnt[c[pn[i]]]] = pn[i];
    cn[p[0]] = 0;
    classes = 1;
    for (int i = 1; i < n; i++) {
        pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
        pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
        if (cur != prev)
            ++classes;
        cn[p[i]] = classes - 1;
    }
    c.swap(cn);
}
return p;
```

The algorithm requires $O(n \log n)$ time and $O(n)$ memory. For simplicity we used the complete ASCII range as alphabet.

If it is known that the string only contains a subset of characters, e.g. only lowercase letters, then the implementation can be optimized, but the optimization factor would likely be insignificant, as the size of the alphabet only matters on the first iteration. Every other iteration depends on

the number of equivalence classes, which may quickly reach $O(n)$ even if initially it was a string over the alphabet of size 2.

Also note, that this algorithm only sorts the cycle shifts. As mentioned at the beginning of this section we can generate the sorted order of the suffixes by appending a character that is smaller than all other characters of the string, and sorting this resulting string by cycle shifts, e.g. by sorting the cycle shifts of $s + \$$. This will obviously give the suffix array of s , however prepended with $|s|$.

```
vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}
```

Applications

Finding the smallest cyclic shift

The algorithm above sorts all cyclic shifts (without appending a character to the string), and therefore $p[0]$ gives the position of the smallest cyclic shift.

Finding a substring in a string

The task is to find a string s inside some text t online - we know the text t beforehand, but not the string s . We can create the suffix array for the text t in $O(|t| \log |t|)$ time. Now we can look for the substring s in the following way. The occurrence of s must be a prefix of some suffix from t . Since we sorted all the suffixes we can perform a binary search for s in p . Comparing the current suffix and the substring s within the binary search can be done in $O(|s|)$ time, therefore the complexity for finding the substring is $O(|s| \log |t|)$. Also notice that if the substring occurs multiple times in t , then all occurrences will be next to each other in p . Therefore the number of occurrences can be found with a second binary search, and all occurrences can be printed easily.

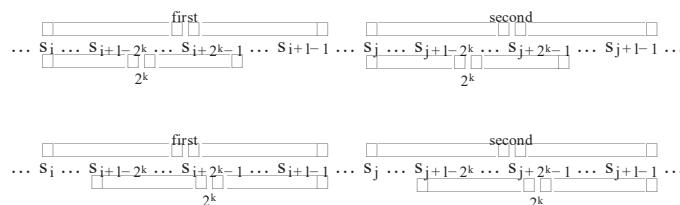
Comparing two substrings of a string

We want to be able to compare two substrings of the same length of a given string s in $O(1)$ time, i.e. checking if the first substring is smaller than the second one.

For this we construct the suffix array in $O(|s| \log |s|)$ time and store all the intermediate results of the equivalence classes $c[]$.

Using this information we can compare any two substring whose length is equal to a power of two in $O(1)$: for this it is sufficient to compare the equivalence classes of both substrings. Now we want to generalize this method to substrings of arbitrary length.

Let's compare two substrings of length l with the starting indices i and j . We find the largest length of a block that is placed inside a substring of this length: the greatest k such that $2^k \leq l$. Then comparing the two substrings can be replaced by comparing two overlapping blocks of length 2^k : first you need to compare the two blocks starting at i and j , and if these are equal then compare the two blocks ending in positions $i + l - 1$ and $j + l - 1$:



Here is the implementation of the comparison. Note that it is assumed that the function gets called with the already calculated k . k can be computed with $\lfloor \log l \rfloor$, but it is more efficient to precompute all k values for every l . See for instance the article about the [Sparse Table](#), which uses a similar idea and computes all $\log$ values.

```
int compare(int i, int j, int l, int k) {
    pair<int, int> a = {c[k][i], c[k][(i+l-(1 << k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+l-(1 << k))%n]};
}
```

```

    return a == b ? 0 : a < b ? -1 : 1;
}

```

Longest common prefix of two substrings with additional memory

For a given string s we want to compute the longest common prefix (**LCP**) of two arbitrary suffixes with position i and j .

The method described here uses $O(|s| \log |s|)$ additional memory. A completely different approach that will only use a linear amount of memory is described in the next section.

We construct the suffix array in $O(|s| \log |s|)$ time, and remember the intermediate results of the arrays $c[]$ from each iteration.

Let's compute the LCP for two suffixes starting at i and j . We can compare any two substrings with a length equal to a power of two in $O(1)$. To do this, we compare the strings by power of twos (from highest to lowest power) and if the substrings of this length are the same, then we add the equal length to the answer and continue checking for the LCP to the right of the equal part, i.e. i and j get added by the current power of two.

```

int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
    return ans;
}

```

Here `log_n` denotes a constant that is equal to the logarithm of n in base 2 rounded down.

Longest common prefix of two substrings without additional memory

We have the same task as in the previous section. We have compute the longest common prefix (**LCP**) for two suffixes of a string s .

Unlike the previous method this one will only use $O(|s|)$ memory. The result of the preprocessing will be an array (which itself is an important source of information about the string, and therefore also used to solve other tasks). LCP queries can be answered by performing RMQ queries (range minimum queries) in this array, so for different implementations it is possible to achieve logarithmic and even constant query time.

The basis for this algorithm is the following idea: we will compute the longest common prefix for each **pair of adjacent suffixes in the sorted order**. In other words we construct an array `lcp[0 ... n - 2]`, where `lcp[i]` is equal to the length of the longest common prefix of the suffixes starting at `p[i]` and `p[i + 1]`. This array will give us an answer for any two adjacent suffixes of the string. Then the answer for arbitrary two suffixes, not necessarily neighboring ones, can be obtained from this array. In fact, let the request be to compute the LCP of the suffixes `p[i]` and `p[j]`. Then the answer to this query will be $\min(lcp[i], lcp[i + 1], \dots, lcp[j - 1])$.

Thus if we have such an array `lcp`, then the problem is reduced to the **RMQ**, which has many wide number of different solutions with different complexities.

So the main task is to **build** this array `lcp`. We will use **Kasai's algorithm**, which can compute this array in $O(n)$ time.

Let's look at two adjacent suffixes in the sorted order (order of the suffix array). Let their starting positions be i and j and their `lcp` equal to $k > 0$. If we remove the first letter of both suffixes - i.e. we take the suffixes $i + 1$ and $j + 1$ - then it should be obvious that the `lcp` of these two is $k - 1$. However we cannot use this value and write it in the `lcp` array, because these two suffixes might not be next to each other in the sorted order. The suffix $i + 1$ will of course be smaller than the suffix $j + 1$, but there might be some suffixes between them. However, since we know that the LCP between two suffixes is the minimum value of all transitions, we also know that the LCP between any two pairs in that interval has to be at least $k - 1$, especially also between $i + 1$ and the next suffix. And possibly it can be bigger.

Now we already can implement the algorithm. We will iterate over the suffixes in order of their length. This way we can reuse the last value k , since going from suffix i to the suffix $i + 1$ is exactly the same as removing the first letter. We will need an additional array `rank`, which will give us the position of a suffix in the sorted list of suffixes.

```

vector<int> lcp_construction(string const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);

```

```

for (int i = 0; i < n; i++)
    rank[p[i]] = i;

int k = 0;
vector<int> lcp(n-1, 0);
for (int i = 0; i < n; i++) {
    if (rank[i] == n - 1) {
        k = 0;
        continue;
    }
    int j = p[rank[i] + 1];
    while (i + k < n && j + k < n && s[i+k] == s[j+k])
        k++;
    lcp[rank[i]] = k;
    if (k)
        k--;
}
return lcp;
}

```

It is easy to see, that we decrease k at most $O(n)$ times (each iteration at most once, except for $\text{rank}[i] = n - 1$, where we directly reset it to 0), and the LCP between two strings is at most $n - 1$, we will also increase k only $O(n)$ times. Therefore the algorithm runs in $O(n)$ time.

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

To do this, we will think about which **new** substrings begin at position $p[0]$, then at $p[1]$, etc. In fact we take the suffixes in sorted order and see what prefixes give new substrings. Thus we will not overlook any by accident.

Because the suffixes are sorted, it is clear that the current suffix $p[i]$ will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix $p[i - 1]$. Thus, all its prefixes except the first $\text{lcp}[i - 1]$ one. Since the length of the current suffix is $n - p[i]$, $n - p[i] - \text{lcp}[i - 1]$ new prefixes start at $p[i]$. Summing over all the suffixes, we get the final answer:

$$\sum_{i=0}^{n-1} (n - p[i]) - \sum_{i=0}^{n-2} \text{lcp}[i] = \frac{n^2 + n}{2} - \sum_{i=0}^{n-2} \text{lcp}[i]$$

Practice Problems

- [Uva 760 - DNA Sequencing](#)
- [Uva 1223 - Editor](#)
- [Codechef - Tandem](#)
- [Codechef - Substrings and Repetitions](#)
- [Codechef - Entangled Strings](#)
- [Codeforces - Martian Strings](#)
- [Codeforces - Little Elephant and Strings](#)
- [SPOJ - Ada and Terramorphing](#)
- [SPOJ - Ada and Substring](#)
- [UVA - 1227 - The longest constant gene](#)
- [SPOJ - Longest Common Substring](#)
- [UVA 11512 - GATTACA](#)
- [LA 7502 - Suffixes and Palindromes](#)
- [GYM - Por Costel and the Censorship Committee](#)
- [UVA 1254 - Top 10](#)
- [UVA 12191 - File Recover](#)
- [UVA 12206 - Stammering Aliens](#)

- [Codechef - Jarvis and LCP](#)
- [LA 3943 - Liking's Letter](#)
- [UVA 11107 - Life Forms](#)
- [UVA 12974 - Exquisite Strings](#)
- [UVA 10526 - Intellectual Property](#)
- [UVA 12338 - Anti-Rhyme Pairs](#)
- [UVA 12191 - File Recover](#)
- [SPOJ - Suffix Array](#)
- [LA 4513 - Stammering Aliens](#)
- [SPOJ - LCS2](#)
- [Codeforces - Fake News \(hard\)](#)
- [SPOJ - Longest Common Substring](#)
- [SPOJ - Lexicographical Substring Search](#)
- [Codeforces - Forbidden Indices](#)
- [Codeforces - Tricky and Clever Password](#)
- [LA 6856 - Circle of digits](#)

Contributors:

[Jakob Kogler](#) (74.88%) [samsidx](#) (8.85%) [Morass](#) (5.26%) [Wanderley da Silva](#) (2.15%) [Anton Ålgmyr](#) (1.91%) [Oleksandr Kulkov](#) (1.91%) [Nhan Tran](#) (1.67%)
[Chirag Agarwal](#) (1.2%) [Vatsal Sharma](#) (0.96%) [Bhuvnesh Jain](#) (0.24%) [Dao Le Bao Minh](#) (0.24%) [Donald Dong](#) (0.24%) [Le Bao Hiep](#) (0.24%)
[Shyamal Vadera](#) (0.24%)

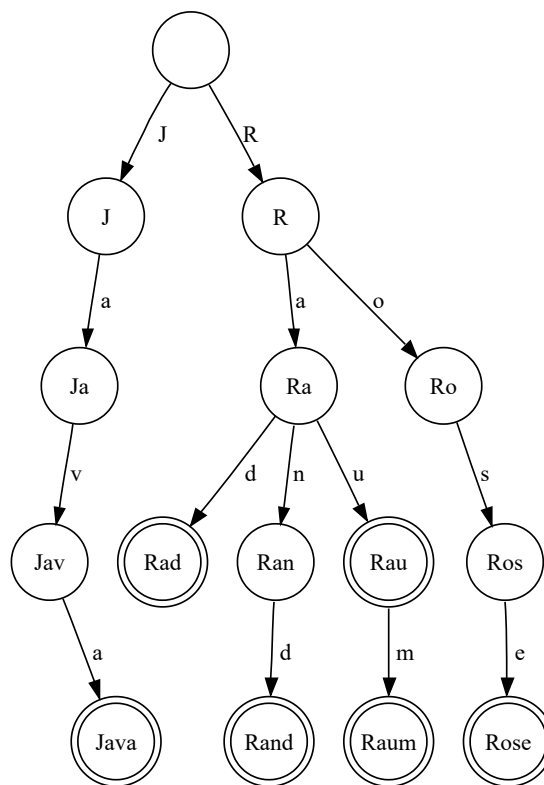
Translated From: e-maxx.ru

Aho-Corasick algorithm

The Aho-Corasick algorithm allows us to quickly search for multiple patterns in a text. The set of pattern strings is also called a *dictionary*. We will denote the total length of its constituent strings by m and the size of the alphabet by k . The algorithm constructs a finite state automaton based on a trie in $O(mk)$ time and then uses it to process the text.

The algorithm was proposed by Alfred Aho and Margaret Corasick in 1975.

Construction of the trie



A trie based on words "Java", "Rad", "Rand", "Rau", "Raum" and "Rose".

The image by [nd](<https://de.wikipedia.org/wiki/Benutzer:Nd>) is distributed under CC BY-SA 3.0 license.

Formally, a trie is a rooted tree, where each edge of the tree is labeled with some letter and outgoing edges of a vertex have distinct labels.

We will identify each vertex in the trie with the string formed by the labels on the path from the root to that vertex.

Each vertex will also have a flag `output` which will be set if the vertex corresponds to a pattern in the dictionary.

Accordingly, a trie for a set of strings is a trie such that each `output` vertex corresponds to one string from the set, and conversely, each string of the set corresponds to one `output` vertex.

We now describe how to construct a trie for a given set of strings in linear time with respect to their total length.

We introduce a structure for the vertices of the tree:

```

const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);

```

Here, we store the trie as an array of `Vertex`. Each `Vertex` contains the flag `output` and the edges in the form of an array `next[]`, where `next[i]` is the index of the vertex that we reach by following the character `i`, or `-1` if there is no such edge. Initially, the trie consists of only one vertex - the root - with the index `0`.

Now we implement a function that will add a string `s` to the trie. The implementation is simple: we start at the root node, and as long as there are edges corresponding to the characters of `s` we follow them. If there is no edge for one character, we generate a new vertex and connect it with an edge. At the end of the process we mark the last vertex with the flag `output`.

```

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}

```

This implementation obviously runs in linear time, and since every vertex stores `k` links, it will use $O(mk)$ memory.

It is possible to decrease the memory consumption to $O(m)$ by using a map instead of an array in each vertex. However, this will increase the time complexity to $O(m \log k)$.

Construction of an automaton

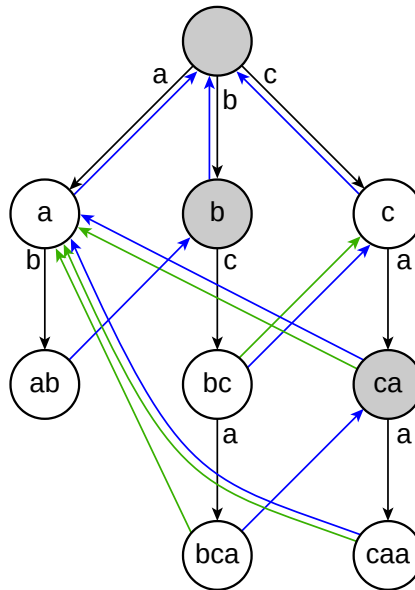
Suppose we have built a trie for the given set of strings. Now let's look at it from a different side. If we look at any vertex, the string that corresponds to it is a prefix of one or more strings in the set, thus each vertex of the trie can be interpreted as a position in one or more strings from the set.

In fact, the trie vertices can be interpreted as states in a **finite deterministic automaton**. From any state we can transition - using some input letter - to other states, i.e., to another position in the set of strings. For example, if there is only one string `abc` in the dictionary, and we are standing at vertex `ab`, then using the letter `c` we can go to the vertex `abc`.

Thus we can understand the edges of the trie as transitions in an automaton according to the corresponding letter. However, in an automaton we need to have transitions for each combination of a state and a letter. If we try to perform a transition using a letter, and there is no corresponding edge in the trie, then we nevertheless must go into some state.

More precisely, suppose we are in a state corresponding to a string `t`, and we want to transition to a different state using the character `c`. If there is an edge labeled with this letter `c`, then we can simply go over this edge, and get the vertex corresponding to `t + c`. If there is no such edge, since we want to maintain the invariant that the current state is the longest partial match in the processed string, we must find the longest string in the trie that's a proper suffix of the string `t`, and try to perform a transition from there.

For example, let the trie be constructed by the strings `ab` and `bc`, and we are currently at the vertex corresponding to `ab`, which is also an output vertex. To transition with the letter `c`, we are forced to go to the state corresponding to the string `b`, and from there follow the edge with the letter `c`.



An Aho-Corasick automaton based on words "a", "ab", "bc", "bca", "c" and "caa".
Blue arrows are suffix links, green arrows are terminal links.

A **suffix link** for a vertex p is an edge that points to the longest proper suffix of the string corresponding to the vertex p . The only special case is the root of the trie, whose suffix link will point to itself. Now we can reformulate the statement about the transitions in the automaton like this: while there is no transition from the current vertex of the trie using the current letter (or until we reach the root), we follow the suffix link.

Thus we reduced the problem of constructing an automaton to the problem of finding suffix links for all vertices of the trie. However, we will build these suffix links, oddly enough, using the transitions constructed in the automaton.

The suffix links of the root vertex and all its immediate children point to the root vertex. For any vertex v deeper in the tree, we can calculate the suffix link as follows: if p is the ancestor of v with c being the letter labeling the edge from p to v , go to p , then follow its suffix link, and perform the transition with the letter c from there.

Thus, the problem of finding the transitions has been reduced to the problem of finding suffix links, and the problem of finding suffix links has been reduced to the problem of finding a suffix link and a transition, except for vertices closer to the root. So we have a recursive dependence that we can resolve in linear time.

Let's move to the implementation. Note that we now will store the ancestor p and the character pch of the edge from p to v for each vertex v . Also, at each vertex we will store the suffix link $link$ (or -1 if it hasn't been calculated yet), and in the array $go[k]$ the transitions in the machine for each symbol (again -1 if it hasn't been calculated yet).

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
```

```

        t[v].next[c] = t.size();
        t.emplace_back(v, ch);
    }
    v = t[v].next[c];
}
t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    }
    return t[v].go[c];
}

```

It is easy to see that thanks to memoization of the suffix links and transitions, the total time for finding all suffix links and transitions will be linear.

For an illustration of the concept refer to slide number 103 of the [Stanford slides](#).

BFS-based construction

Instead of computing transitions and suffix links with recursive calls to `go` and `get_link`, it is possible to compute them bottom-up starting from the root. (In fact, when the dictionary consists of only one string, we obtain the familiar Knuth-Morris-Pratt algorithm.)

This approach will have some advantages over the one described above as, instead of the total length m , its running time depends only on the number of vertices n in the trie. Moreover, it is possible to adapt it for large alphabets using a persistent array data structure, thus making the construction time $O(n \log k)$ instead of $O(mk)$, which is a significant improvement granted that m may go up to n^2 .

We can reason inductively using the fact that BFS from the root traverses vertices in order of increasing length. We may assume that when we're in a vertex v , its suffix link $u = \text{link}[v]$ is already successfully computed, and for all vertices with shorter length transitions from them are also fully computed.

Assume that at the moment we stand in a vertex v and consider a character c . We essentially have two cases:

1. $\text{go}[v][c] = -1$. In this case, we may assign $\text{go}[v][c] = \text{go}[u][c]$, which is already known by the induction hypothesis;
2. $\text{go}[v][c] = w \neq -1$. In this case, we may assign $\text{link}[w] = \text{go}[u][c]$.

In this way, we spend $O(1)$ time per each pair of a vertex and a character, making the running time $O(nk)$. The major overhead here is that we copy a lot of transitions from u in the first case, while the transitions of the second case form the trie and sum up to n over all vertices. To avoid the copying of $\text{go}[u][c]$, we may use a persistent array data structure, using which we initially copy $\text{go}[u]$ into $\text{go}[v]$ and then only update values for characters in which the transition would differ. This leads to the $O(n \log k)$ algorithm.

Applications

Find all strings from a given set in a text

We are given a set of strings and a text. We have to print all occurrences of all strings from the set in the given text in $O(\text{len} + \text{ans})$, where len is the length of the text and ans is the size of the answer.

We construct an automaton for this set of strings. We will now process the text letter by letter using the automaton, starting at the root of the trie. If we are at any time at state v , and the next letter is c , then we transition to the next state with $go(v, c)$, thereby either increasing the length of the current match substring by 1, or decreasing it by following a suffix link.

How can we find out for a state v , if there are any matches with strings for the set? First, it is clear that if we stand on a `output` vertex, then the string corresponding to the vertex ends at this position in the text. However this is by no means the only possible case of achieving a match: if we can reach one or more `output` vertices by moving along the suffix links, then there will be also a match corresponding to each found `output` vertex. A simple example demonstrating this situation can be created using the set of strings $\{dabce, ab, bc\}$ and the text `dabc`.

Thus if we store in each `output` vertex the index of the string corresponding to it (or the list of indices if duplicate strings appear in the set), then we can find in $O(n)$ time the indices of all strings which match the current state, by simply following the suffix links from the current vertex to the root. This is not the most efficient solution, since this results in $O(n \cdot len)$ complexity overall. However, this can be optimized by computing and storing the nearest `output` vertex that is reachable using suffix links (this is sometimes called the **exit link**). This value we can compute lazily in linear time. Thus for each vertex we can advance in $O(1)$ time to the next marked vertex in the suffix link path, i.e. to the next match. Thus for each match we spend $O(1)$ time, and therefore we reach the complexity $O(len + ans)$.

If you only want to count the occurrences and not find the indices themselves, you can calculate the number of marked vertices in the suffix link path for each vertex v . This can be calculated in $O(n)$ time in total. Thus we can sum up all matches in $O(len)$.

Finding the lexicographically smallest string of a given length that doesn't match any given strings

A set of strings and a length L is given. We have to find a string of length L , which does not contain any of the strings, and derive the lexicographically smallest of such strings.

We can construct the automaton for the set of strings. Recall that `output` vertices are the states where we have a match with a string from the set. Since in this task we have to avoid matches, we are not allowed to enter such states. On the other hand we can enter all other vertices. Thus we delete all "bad" vertices from the machine, and in the remaining graph of the automaton we find the lexicographically smallest path of length L . This task can be solved in $O(L)$ for example by [depth first search](#).

Finding the shortest string containing all given strings

Here we use the same ideas. For each vertex we store a mask that denotes the strings which match at this state. Then the problem can be reformulated as follows: initially being in the state ($v = \text{root}, \text{mask} = 0$), we want to reach the state ($v, \text{mask} = 2^n - 1$), where n is the number of strings in the set. When we transition from one state to another using a letter, we update the mask accordingly. By running a [breadth first search](#) we can find a path to the state ($v, \text{mask} = 2^n - 1$) with the smallest length.

Finding the lexicographically smallest string of length L containing k strings

As in the previous problem, we calculate for each vertex the number of matches that correspond to it (that is the number of marked vertices reachable using suffix links). We reformulate the problem: the current state is determined by a triple of numbers ($v, \text{len}, \text{cnt}$), and we want to reach from the state ($\text{root}, 0, 0$) the state (v, L, k), where v can be any vertex. Thus we can find such a path using depth first search (and if the search looks at the edges in their natural order, then the found path will automatically be the lexicographically smallest).

Problems

- [UVA #11590 - Prefix Lookup](#)
- [UVA #11171 - SMS](#)
- [UVA #10679 - I Love Strings!!](#)
- [Codeforces - x-prime Substrings](#)
- [Codeforces - Frequency of String](#)
- [CodeChef - TWOSTRS](#)

References

- [Stanford's CS166 - Aho-Corasick Automata \(Condensed\)](#)

Contributors:

[Jakob Kogler](#) (61.07%) [Wiktor Kuchta](#) (23.93%) [Oleksandr Kulkov](#) (11.78%) [Madhur Chauhan](#) (1.79%) [Aman Kumar Singh](#) (0.36%)
[Gabriel Fernandes de Oliveira](#) (0.36%) [Gaurang Tandon](#) (0.36%) [Jay Wang](#) (0.36%)

Translated From: e-maxx.ru

Suffix Tree. Ukkonen's Algorithm

This article is a stub and doesn't contain any descriptions. For a description of the algorithm, refer to other sources, such as [Algorithms on Strings, Trees, and Sequences](#) by Dan Gusfield.

This algorithm builds a suffix tree for a given string s of length n in $O(n \log(k))$ time, where k is the size of the alphabet (if k is considered to be a constant, the asymptotic behavior is linear).

The input to the algorithm are the string s and its length n , which are passed as global variables.

The main function `build_tree` builds a suffix tree. It is stored as an array of structures `node`, where `node[0]` is the root of the tree.

In order to simplify the code, the edges are stored in the same structures: for each vertex its structure `node` stores the information about the edge between it and its parent. Overall each `node` stores the following information:

- `l, r` - left and right boundaries of the substring `s[l..r-1]` which correspond to the edge to this node,
- `par` - the parent node,
- `link` - the suffix link,
- `next` - the list of edges going out from this node.

```
string s;
int n;

struct node {
    int l, r, par, link;
    map<char,int> next;

    node (int l=0, int r=0, int par=-1)
        : l(l), r(r), par(par), link(-1) {}
    int len() { return r - l; }
    int &get (char c) {
        if (!next.count(c)) next[c] = -1;
        return next[c];
    }
};
node t[MAXN];
int sz;

struct state {
    int v, pos;
    state (int v, int pos) : v(v), pos(pos) {}
};
state ptr (0, 0);

state go (state st, int l, int r) {
    while (l < r)
        if (st.pos == t[st.v].len()) {
            st = state (t[st.v].get( s[l] ), 0);
            if (st.v == -1) return st;
        }
        else {
            if (s[ t[st.v].l + st.pos ] != s[l])
                return state (-1, -1);
            if (r-1 < t[st.v].len() - st.pos)
                return state (st.v, st.pos + r-1);
            l += t[st.v].len() - st.pos;
            st.pos = t[st.v].len();
        }
    return st;
}

int split (state st) {
    if (st.pos == t[st.v].len())
```

```

        return st.v;
    if (st.pos == 0)
        return t[st.v].par;
    node v = t[st.v];
    int id = sz++;
    t[id] = node (v.l, v.l+st.pos, v.par);
    t[v.par].get( s[v.l] ) = id;
    t[id].get( s[v.l+st.pos] ) = st.v;
    t[st.v].par = id;
    t[st.v].l += st.pos;
    return id;
}

int get_link (int v) {
    if (t[v].link != -1) return t[v].link;
    if (t[v].par == -1) return 0;
    int to = get_link (t[v].par);
    return t[v].link = split (go (state(to,t[to].len()), t[v].l + (t[v].par==0), t[v].r));
}

void tree_extend (int pos) {
    for(;;) {
        state nptr = go (ptr, pos, pos+1);
        if (nptr.v != -1) {
            ptr = nptr;
            return;
        }

        int mid = split (ptr);
        int leaf = sz++;
        t[leaf] = node (pos, n, mid);
        t[mid].get( s[pos] ) = leaf;

        ptr.v = get_link (mid);
        ptr.pos = t[ptr.v].len();
        if (!mid) break;
    }
}

void build_tree() {
    sz = 1;
    for (int i=0; i<n; ++i)
        tree_extend (i);
}

```

Compressed Implementation

This compressed implementation was proposed by [freopen](#).

```

const int N=1000000, INF=1000000000;
string a;
int t[N][26], l[N], r[N], p[N], s[N], tv, tp, ts, la;

void ukkadd (int c) {
    suff++;
    if (r[tv]<tp) {
        if (t[tv][c]==-1) { t[tv][c]=ts; l[ts]=la;
            p[ts++]=tv; tv=s[tv]; tp=r[tv]+1; goto suff; }
        tv=t[tv][c]; tp=l[tv];
    }
    if (tp==-1 || c==a[tp]-'a') tp++; else {
        l[ts+1]=la; p[ts+1]=ts;
        l[ts]=l[tv]; r[ts]=tp-1; p[ts]=p[tv]; t[ts][c]=ts+1; t[ts][a[tp]-'a']=tv;
        l[tv]=tp; p[tv]=ts; t[p[ts]][a[l[ts]]-'a']=ts; ts+=2;
        tv=s[p[ts-2]]; tp=l[ts-2];
        while (tp<=r[ts-2]) { tv=t[tv][a[tp]-'a']; tp+=r[tv]-l[tv]+1;}
        if (tp==r[ts-2]+1) s[ts-2]=tv; else s[ts-2]=ts;
        tp=r[tv]-(tp-r[ts-2])+2; goto suff;
    }
}

void build() {
    ts=2;
}

```

```

    tv=0;
    tp=0;
    fill(r,r+N,(int)a.size()-1);
    s[0]=1;
    l[0]=-1;
    r[0]=-1;
    l[1]=-1;
    r[1]=-1;
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+26,0);
    for (la=0; la<(int)a.size(); ++la)
        ukkadd (a[la]-'a');
}

```

Same code with comments:

```

const int N=1000000, // maximum possible number of nodes in suffix tree
INF=1000000000; // infinity constant
string a; // input string for which the suffix tree is being built
int t[N][26], // array of transitions (state, letter)
l[N], // left...
r[N], // ...and right boundaries of the substring of a which correspond to incoming edge
p[N], // parent of the node
s[N], // suffix link
tv, // the node of the current suffix (if we're mid-edge, the lower node of the edge)
tp, // position in the string which corresponds to the position on the edge (between l[tv] and r[tv], inclusive)
ts, // the number of nodes
la; // the current character in the string

void ukkadd(int c) { // add character s to the tree
    suff; // we'll return here after each transition to the suffix (and will add character again)
    if (r[tv]<tp) { // check whether we're still within the boundaries of the current edge
        // if we're not, find the next edge. If it doesn't exist, create a leaf and add it to the tree
        if (t[tv][c]==-1) {t[tv][c]=ts;l[ts]=la;p[ts++]=tv;tv=s[tv];tp=r[tv]+1;goto suff;}
        tv=t[tv][c];tp=l[tv];
    } // otherwise just proceed to the next edge
    if (tp==-1 || c==a[tp]-'a')
        tp++; // if the letter on the edge equal c, go down that edge
    else {
        // otherwise split the edge in two with middle in node ts
        l[ts]=l[tv];r[ts]=tp-1;p[ts]=p[tv];t[ts][a[tp]-'a']=tv;
        // add leaf ts+1. It corresponds to transition through c.
        t[ts][c]=ts+1;l[ts+1]=la;p[ts+1]=ts;
        // update info for the current node - remember to mark ts as parent of tv
        l[tv]=tp;p[tv]=ts;t[p[ts]][a[l[ts]]-'a']=ts;ts+=2;
        // prepare for descent
        // tp will mark where are we in the current suffix
        tv=s[p[ts-2]];tp=l[ts-2];
        // while the current suffix is not over, descend
        while (tp<=r[ts-2]) {tv=t[tv][a[tp]-'a'];tp+=r[tv]-l[tv]+1;}
        // if we're in a node, add a suffix link to it, otherwise add the link to ts
        // (we'll create ts on next iteration).
        if (tp==r[ts-2]+1) s[ts-2]=tv; else s[ts-2]=ts;
        // add tp to the new edge and return to add letter to suffix
        tp=r[tv]-(tp-r[ts-2])+2;goto suff;
    }
}

void build() {
    ts=2;
    tv=0;
    tp=0;
    fill(r,r+N,(int)a.size()-1);
    // initialize data for the root of the tree
    s[0]=1;
    l[0]=-1;
    r[0]=-1;
    l[1]=-1;
    r[1]=-1;
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+26,0);
    // add the text to the tree, letter by letter
    for (la=0; la<(int)a.size(); ++la)
        ukkadd (a[la]-'a');
}

```

Practice Problems

- [UVA 10679 - I Love Strings!!!](#)

Contributors:

[Chirag Agarwal](#) (59.82%) [Mariia Mykhailova](#) (37.05%) [Oleksandr Kulkov](#) (2.68%) [Dustin Miao](#) (0.45%)

Translated From: e-maxx.ru

Suffix Automaton

A **suffix automaton** is a powerful data structure that allows solving many string-related problems.

For example, you can search for all occurrences of one string in another, or count the amount of different substrings of a given string. Both tasks can be solved in linear time with the help of a suffix automaton.

Intuitively a suffix automaton can be understood as a compressed form of **all substrings** of a given string. An impressive fact is, that the suffix automaton contains all this information in a highly compressed form. For a string of length n it only requires $O(n)$ memory. Moreover, it can also be built in $O(n)$ time (if we consider the size k of the alphabet as a constant), otherwise both the memory and the time complexity will be $O(n \log k)$.

The linearity of the size of the suffix automaton was first discovered in 1983 by Blumer et al., and in 1985 the first linear algorithms for the construction was presented by Crochemore and Blumer.

Definition of a suffix automaton

A suffix automaton for a given string s is a minimal **DFA** (deterministic finite automaton / deterministic finite state machine) that accepts all the suffixes of the string s .

In other words:

- A suffix automaton is an oriented acyclic graph. The vertices are called **states**, and the edges are called **transitions** between states.
- One of the states t_0 is the **initial state**, and it must be the source of the graph (all other states are reachable from t_0).
- Each **transition** is labeled with some character. All transitions originating from a state must have **different** labels.
- One or multiple states are marked as **terminal states**. If we start from the initial state t_0 and move along transitions to a terminal state, then the labels of the passed transitions must spell one of the suffixes of the string s . Each of the suffixes of s must be spellable using a path from t_0 to a terminal state.
- The suffix automaton contains the minimum number of vertices among all automata satisfying the conditions described above.

Substring property

The simplest and most important property of a suffix automaton is, that it contains information about all substrings of the string s . Any path starting at the initial state t_0 , if we write down the labels of the transitions, forms a **substring** of s . And conversely every substring of s corresponds to a certain path starting at t_0 .

In order to simplify the explanations, we will say that the substring **corresponds** to that path (starting at t_0 and the labels spell the substring). And conversely we say that any path **corresponds** to the string spelled by its labels.

One or multiple paths can lead to a state. Thus, we will say that a state **corresponds** to the set of strings, which correspond to these paths.

Examples of constructed suffix automata

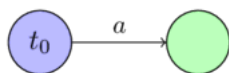
Here we will show some examples of suffix automata for several simple strings.

We will denote the initial state with blue and the terminal states with green.

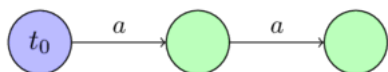
For the string $s = ""$:



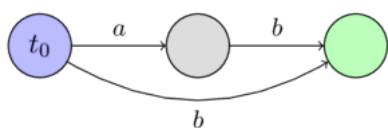
For the string $s = "a"$:



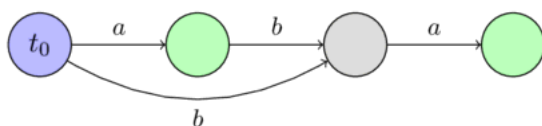
For the string $s = "aa"$:



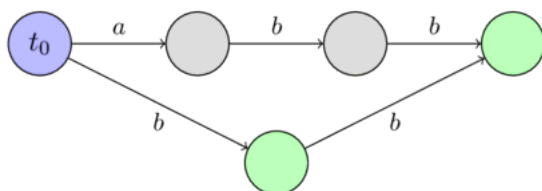
For the string $s = "ab"$:



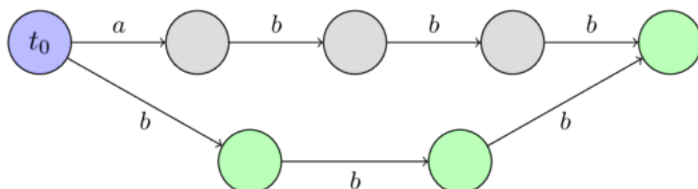
For the string $s = "aba"$:



For the string $s = "abb"$:



For the string $s = "abbb"$:



Construction in linear time

Before we describe the algorithm to construct a suffix automaton in linear time, we need to introduce several new concepts and simple proofs, which will be very important in understanding the construction.

End positions endpos

Consider any non-empty substring t of the string s . We will denote with $\text{endpos}(t)$ the set of all positions in the string s , in which the occurrences of t end. For instance, we have $\text{endpos}("bc") = \{2, 4\}$ for the string "abcbc".

We will call two substrings t_1 and t_2 endpos -equivalent, if their ending sets coincide: $\text{endpos}(t_1) = \text{endpos}(t_2)$. Thus all non-empty substrings of the string s can be decomposed into several **equivalence classes** according to their sets endpos .

It turns out, that in a suffix machine endpos -equivalent substrings **correspond to the same state**. In other words the number of states in a suffix automaton is equal to the number of equivalence classes among all substrings, plus the initial state. Each state of a suffix automaton corresponds to one or more substrings having the same value endpos .

We will later describe the construction algorithm using this assumption. We will then see, that all the required properties of a suffix automaton, except for the minimality, are fulfilled. And the minimality follows from Nerode's theorem (which will not be proven in this article).

We can make some important observations concerning the values endpos :

Lemma 1: Two non-empty substrings u and w (with $\text{length}(u) \leq \text{length}(w)$) are endpos -equivalent, if and only if the string u occurs in s only in the form of a suffix of w .

The proof is obvious. If u and w have the same endpos values, then u is a suffix of w and appears only in the form of a suffix of w in s . And if u is a suffix of w and appears only in the form as a suffix in s , then the values endpos are equal by definition.

Lemma 2: Consider two non-empty substrings u and w (with $\text{length}(u) \leq \text{length}(w)$). Then their sets endpos either don't intersect at all, or $\text{endpos}(w)$ is a subset of $\text{endpos}(u)$. And it depends on if u is a suffix of w or not.

$$\begin{cases} \text{endpos}(w) \subseteq \text{endpos}(u) & \text{if } u \text{ is a suffix of } w \\ \text{endpos}(w) \cap \text{endpos}(u) = \emptyset & \text{otherwise} \end{cases}$$

Proof: If the sets $\text{endpos}(u)$ and $\text{endpos}(w)$ have at least one common element, then the strings u and w both end in that position, i.e. u is a suffix of w . But then at every occurrence of w also appears the substring u , which means that $\text{endpos}(w)$ is a subset of $\text{endpos}(u)$.

Lemma 3: Consider an endpos -equivalence class. Sort all the substrings in this class by decreasing length. Then in the resulting sequence each substring will be one shorter than the previous one, and at the same time will be a suffix of the previous one. In other words, in a same equivalence class, the shorter substrings are actually suffixes of the longer substrings, and they take all possible lengths in a certain interval $[x; y]$.

Proof: Fix some endpos -equivalence class. If it only contains one string, then the lemma is obviously true. Now let's say that the number of strings in the class is greater than one.

According to Lemma 1, two different endpos -equivalent strings are always in such a way, that the shorter one is a proper suffix of the longer one. Consequently, there cannot be two strings of the same length in the equivalence class.

Let's denote by w the longest, and through u the shortest string in the equivalence class. According to Lemma 1, the string u is a proper suffix of the string w . Consider now any suffix of w with a length in the interval $[\text{length}(u); \text{length}(w)]$. It is easy to see, that this suffix is also contained in the same equivalence class. Because this suffix can only appear in the form of a suffix of w in the string s (since also the shorter suffix u occurs in s only in the form of a suffix of w). Consequently, according to Lemma 1, this suffix is endpos -equivalent to the string w .

Suffix links link

Consider some state $v \neq t_0$ in the automaton. As we know, the state v corresponds to the class of strings with the same endpos values. And if we denote by w the longest of these strings, then all the other strings are suffixes of w .

We also know the first few suffixes of a string w (if we consider suffixes in descending order of their length) are all contained in this equivalence class, and all other suffixes (at least one other - the empty suffix) are in some other classes. We denote by t the biggest such suffix, and make a suffix link to it.

In other words, a **suffix link** $\text{link}(v)$ leads to the state that corresponds to the **longest suffix** of w that is in another endpos -equivalence class.

Here we assume that the initial state t_0 corresponds to its own equivalence class (containing only the empty string), and for convenience we set $\text{endpos}(t_0) = \{-1, 0, \dots, \text{length}(s) - 1\}$.

Lemma 4: Suffix links form a **tree** with the root t_0 .

Proof: Consider an arbitrary state $v \neq t_0$. A suffix link $\text{link}(v)$ leads to a state corresponding to strings with strictly smaller length (this follows from the definition of the suffix links and from Lemma 3). Therefore, by moving along the suffix links, we will sooner or later come to the initial state t_0 , which corresponds to the empty string.

Lemma 5: If we construct a tree using the sets endpos (by the rule that the set of a parent node contains the sets of all children as subsets), then the structure will coincide with the tree of suffix links.

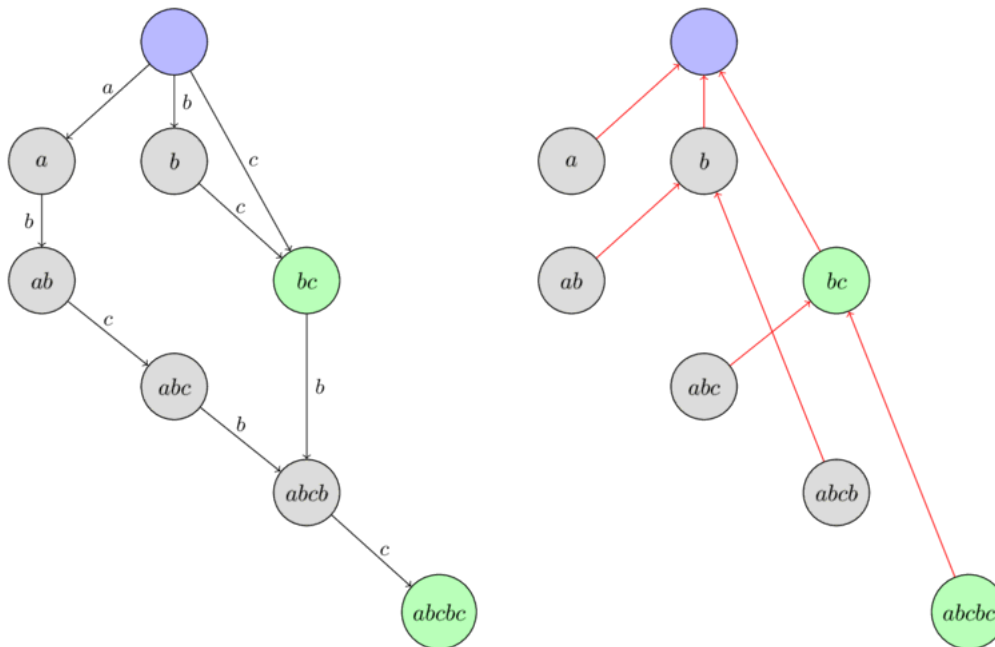
Proof: The fact that we can construct a tree using the sets endpos follows directly from Lemma 2 (that any two sets either do not intersect or one is contained in the other).

Let us now consider an arbitrary state $v \neq t_0$, and its suffix link $\text{link}(v)$. From the definition of the suffix link and from Lemma 2 it follows that

$$\text{endpos}(v) \subseteq \text{endpos}(\text{link}(v)),$$

which together with the previous lemma proves the assertion: the tree of suffix links is essentially a tree of sets endpos .

Here is an **example** of a tree of suffix links in the suffix automaton build for the string "abcbc". The nodes are labeled with the longest substring from the corresponding equivalence class.



Recap

Before proceeding to the algorithm itself, we recap the accumulated knowledge, and introduce a few auxiliary notations.

- The substrings of the string s can be decomposed into equivalence classes according to their end positions endpos .
- The suffix automaton consists of the initial state t_0 , as well as of one state for each endpos -equivalence class.
- For each state v one or multiple substrings match. We denote by $\text{longest}(v)$ the longest such string, and through $\text{len}(v)$ its length. We denote by $\text{shortest}(v)$ the shortest such substring, and its length with $\text{minlen}(v)$. Then all the strings corresponding to this state are different suffixes of the string $\text{longest}(v)$ and have all possible lengths in the interval $[\text{minlen}(v); \text{len}(v)]$.
- For each state $v \neq t_0$ a suffix link is defined as a link, that leads to a state that corresponds to the suffix of the string $\text{longest}(v)$ of length $\text{minlen}(v) - 1$. The suffix links form a tree with the root in t_0 , and at the same time this tree forms an inclusion relationship between the sets endpos .

- We can express $\text{minlen}(v)$ for $v \neq t_0$ using the suffix link $\text{link}(v)$ as:

$$\text{minlen}(v) = \text{len}(\text{link}(v)) + 1$$

- If we start from an arbitrary state v_0 and follow the suffix links, then sooner or later we will reach the initial state t_0 . In this case we obtain a sequence of disjoint intervals $[\text{minlen}(v_i); \text{len}(v_i)]$, which in union forms the continuous interval $[0; \text{len}(v_0)]$.

Algorithm

Now we can proceed to the algorithm itself. The algorithm will be **online**, i.e. we will add the characters of the string one by one, and modify the automaton accordingly in each step.

To achieve linear memory consumption, we will only store the values len , link and a list of transitions in each state. We will not label terminal states (but we will later show how to arrange these labels after constructing the suffix automaton).

Initially the automaton consists of a single state t_0 , which will be the index 0 (the remaining states will receive the indices 1, 2, ...). We assign it $\text{len} = 0$ and $\text{link} = -1$ for convenience (-1 will be a fictional, non-existing state).

Now the whole task boils down to implementing the process of **adding one character** c to the end of the current string. Let us describe this process:

- Let last be the state corresponding to the entire string before adding the character c . (Initially we set $\text{last} = 0$, and we will change last in the last step of the algorithm accordingly.)
- Create a new state cur , and assign it with $\text{len}(\text{cur}) = \text{len}(\text{last}) + 1$. The value $\text{link}(\text{cur})$ is not known at the time.
- Now we do the following procedure: We start at the state last . While there isn't a transition through the letter c , we will add a transition to the state cur , and follow the suffix link. If at some point there already exists a transition through the letter c , then we will stop and denote this state with p .
- If we haven't found such a state p , then we reached the fictitious state -1 , then we can just assign $\text{link}(\text{cur}) = 0$ and leave.
- Suppose now that we have found a state p , from which there exists a transition through the letter c . We will denote the state, to which the transition leads, with q .
- Now we have two cases. Either $\text{len}(p) + 1 = \text{len}(q)$, or not.
- If $\text{len}(p) + 1 = \text{len}(q)$, then we can simply assign $\text{link}(\text{cur}) = q$ and leave.
- Otherwise it is a bit more complicated. It is necessary to **clone** the state q : we create a new state clone , copy all the data from q (suffix link and transition) except the value len . We will assign $\text{len}(\text{clone}) = \text{len}(p) + 1$.

After cloning we direct the suffix link from cur to clone , and also from q to clone .

Finally we need to walk from the state p back using suffix links as long as there is a transition through c to the state q , and redirect all those to the state clone .

- In any of the three cases, after completing the procedure, we update the value last with the state cur .

If we also want to know which states are **terminal** and which are not, we can find all terminal states after constructing the complete suffix automaton for the entire string s . To do this, we take the state corresponding to the entire string (stored in the variable last), and follow its suffix links until we reach the initial state. We will mark all visited states as terminal. It is easy to understand that by doing so we will mark exactly the states corresponding to all the suffixes of the string s , which are exactly the terminal states.

In the next section we will look in detail at each step and show its **correctness**.

Here we only note that, since we only create one or two new states for each character of s , the suffix automaton contains a **linear number of states**.

The linearity of the number of transitions, and in general the linearity of the runtime of the algorithm is less clear, and they will be proven after we proved the correctness.

Correctness

- We will call a transition (p, q) **continuous** if $\text{len}(p) + 1 = \text{len}(q)$. Otherwise, i.e. when $\text{len}(p) + 1 < \text{len}(q)$, the transition will be called **non-continuous**.

As we can see from the description of the algorithm, continuous and non-continuous transitions will lead to different cases of the algorithm. Continuous transitions are fixed, and will never change again. In contrast non-continuous transition may change, when new letters are added to the string (the end of the transition edge may change).

- To avoid ambiguity we will denote the string, for which the suffix automaton was built before adding the current character c , with s .
- The algorithm begins with creating a new state cur , which will correspond to the entire string $s + c$. It is clear why we have to create a new state. Together with the new character a new equivalence class is created.
- After creating a new state we traverse by suffix links starting from the state corresponding to the entire string s . For each state we try to add a transition with the character c to the new state cur . Thus we append to each suffix of s the character c . However we can only add these new transitions, if they don't conflict with an already existing one. Therefore as soon as we find an already existing transition with c we have to stop.
- In the simplest case we reached the fictitious state -1 . This means we added the transition with c to all suffixes of s . This also means, that the character c hasn't been part of the string s before. Therefore the suffix link of cur has to lead to the state 0 .
- In the second case we came across an existing transition (p, q) . This means that we tried to add a string $x + c$ (where x is a suffix of s) to the machine that **already exists** in the machine (the string $x + c$ already appears as a substring of s). Since we assume that the automaton for the string s is built correctly, we should not add a new transition here.

However there is a difficulty. To which state should the suffix link from the state cur lead? We have to make a suffix link to a state, in which the longest string is exactly $x + c$, i.e. the len of this state should be $len(p) + 1$. However it is possible, that such a state doesn't yet exist, i.e. $len(q) > len(p) + 1$. In this case we have to create such a state, by **splitting** the state q .

- If the transition (p, q) turns out to be continuous, then $len(q) = len(p) + 1$. In this case everything is simple. We direct the suffix link from cur to the state q .
- Otherwise the transition is non-continuous, i.e. $len(q) > len(p) + 1$. This means that the state q corresponds to not only the suffix of $s + c$ with length $len(p) + 1$, but also to longer substrings of s . We can do nothing other than **splitting** the state q into two sub-states, so that the first one has length $len(p) + 1$.

How can we split a state? We **clone** the state q , which gives us the state $clone$, and we set $len(clone) = len(p) + 1$. We copy all the transitions from q to $clone$, because we don't want to change the paths that traverse through q . Also we set the suffix link from $clone$ to the target of the suffix link of q , and set the suffix link of q to $clone$.

And after splitting the state, we set the suffix link from cur to $clone$.

In the last step we change some of the transitions to q , we redirect them to $clone$. Which transitions do we have to change? It is enough to redirect only the transitions corresponding to all the suffixes of the string $w + c$ (where w is the longest string of p), i.e. we need to continue to move along the suffix links, starting from the vertex p until we reach the fictitious state -1 or a transition that leads to a different state than q .

Linear number of operations

First we immediately make the assumption that the size of the alphabet is **constant**. If this is not the case, then it will not be possible to talk about the linear time complexity. The list of transitions from one vertex will be stored in a balanced tree, which allows you to quickly perform key search operations and adding keys. Therefore if we denote with k the size of the alphabet, then the asymptotic behavior of the algorithm will be $O(n \log k)$ with $O(n)$ memory. However if the alphabet is small enough, then you can sacrifice memory by avoiding balanced trees, and store the transitions at each vertex as an array of length k (for quick searching by key) and a dynamic list (to quickly traverse all available keys). Thus we reach the $O(n)$ time complexity for the algorithm, but at a cost of $O(nk)$ memory complexity.

So we will consider the size of the alphabet to be constant, i.e. each operation of searching for a transition on a character, adding a transition, searching for the next transition - all these operations can be done in $O(1)$.

If we consider all parts of the algorithm, then it contains three places in the algorithm in which the linear complexity is not obvious:

- The first place is the traversal through the suffix links from the state $last$, adding transitions with the character c .
- The second place is the copying of transitions when the state q is cloned into a new state $clone$.
- Third place is changing the transition leading to q , redirecting them to $clone$.

We use the fact that the size of the suffix automaton (both in the number of states and in the number of transitions) is **linear**. (The proof of the linearity of the number of states is the algorithm itself, and the proof of linearity of the number of states is given below, after the implementation of the algorithm).

Thus the total complexity of the **first and second places** is obvious, after all each operation adds only one amortized new transition to the automaton.

It remains to estimate the total complexity of the **third place**, in which we redirect transitions, that pointed originally to q , to $clone$. We denote $v = \text{longest}(p)$. This is a suffix of the string s , and with each iteration its length decreases - and therefore the position v as the suffix of the string s increases monotonically with each iteration. In this case, if before the first iteration of the loop, the corresponding string v was at the depth k ($k \geq 2$) from $last$ (by counting the depth as the number of suffix links), then after the last iteration the string $v + c$ will be a 2-th suffix link on the path from cur (which will become the new value $last$).

Thus, each iteration of this loop leads to the fact that the position of the string $\text{longest}(\text{link}(\text{link}(last)))$ as a suffix of the current string will monotonically increase. Therefore this cycle cannot be executed more than n iterations, which was required to prove.

Implementation

First we describe a data structure that will store all information about a specific transition (len , $link$ and the list of transitions). If necessary you can add a terminal flag here, as well as other information. We will store the list of transitions in the form of a map, which allows us to achieve total $O(n)$ memory and $O(n \log k)$ time for processing the entire string.

```
struct state {
    int len, link;
    map<char, int> next;
};
```

The suffix automaton itself will be stored in an array of these structures `state`. We store the current size `sz` and also the variable `last`, the state corresponding to the entire string at the moment.

```
const int MAXLEN = 100000;
state st[MAXLEN * 2];
int sz, last;
```

We give a function that initializes a suffix automaton (creating a suffix automaton with a single state).

```
void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}
```

And finally we give the implementation of the main function - which adds the next character to the end of the current line, rebuilding the machine accordingly.

```
void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
}
```

```
    last = cur;
}
```

As mentioned above, if you sacrifice memory ($O(nk)$, where k is the size of the alphabet), then you can achieve the build time of the machine in $O(n)$, even for any alphabet size k . But for this you will have to store an array of size k in each state (for quickly jumping to the transition of the letter), and additionally a list of all transitions (to quickly iterate over the transitions them).

Additional properties

Number of states

The number of states in a suffix automaton of the string s of length n **doesn't exceed** $2n - 1$ (for $n \geq 2$).

The proof is the construction algorithm itself, since initially the automaton consists of one state, and in the first and second iteration only a single state will be created, and in the remaining $n - 2$ steps at most 2 states will be created each.

However we can also **show** this estimation **without knowing the algorithm**. Let us recall that the number of states is equal to the number of different sets endpos . In addition these sets endpos form a tree (a parent vertex contains all children sets in his set). Consider this tree and transform it a little bit: as long as it has an internal vertex with only one child (which means that the set of the child misses at least one position from the parent set), we create a new child with the set of the missing positions. In the end we have a tree in which each inner vertex has a degree greater than one, and the number of leaves does not exceed n . Therefore there are no more than $2n - 1$ vertices in such a tree.

This bound of the number of states can actually be achieved for each n . A possible string is:

"abbb ... bbb"

In each iteration, starting at the third one, the algorithm will split a state, resulting in exactly $2n - 1$ states.

Number of transitions

The number of transitions in a suffix automaton of a string s of length n **doesn't exceed** $3n - 4$ (for $n \geq 3$).

Let us prove this:

Let us first estimate the number of continuous transitions. Consider a spanning tree of the longest paths in the automaton starting in the state t_0 . This skeleton will consist of only the continuous edges, and therefore their number is less than the number of states, i.e. it does not exceed $2n - 2$.

Now let us estimate the number of non-continuous transitions. Let the current non-continuous transition be (p, q) with the character c . We take the correspondent string $u + c + w$, where the string u corresponds to the longest path from the initial state to p , and w to the longest path from q to any terminal state. On one hand, each such string $u + c + w$ for each incomplete strings will be different (since the strings u and w are formed only by complete transitions). On the other hand each such string $u + c + w$, by the definition of the terminal states, will be a suffix of the entire string s . Since there are only n non-empty suffixes of s , and none of the strings $u + c + w$ can contain s (because the entire string only contains complete transitions), the total number of incomplete transitions does not exceed $n - 1$.

Combining these two estimates gives us the bound $3n - 3$. However, since the maximum number of states can only be achieved with the test case "abbb\dots bbb" and this case has clearly less than $3n - 3$ transitions, we get the tighter bound of $3n - 4$ for the number of transitions in a suffix automaton.

This bound can also be achieved with the string:

"abbb ... bbbc"

Applications

Here we look at some tasks that can be solved using the suffix automaton. For the simplicity we assume that the alphabet size k is constant, which allows us to consider the complexity of appending a character and the traversal as constant.

Check for occurrence

Given a text T , and multiple patterns P . We have to check whether or not the strings P appear as a substring of T .

We build a suffix automaton of the text T in $O(\text{length}(T))$ time. To check if a pattern P appears in T , we follow the transitions, starting from t_0 , according to the characters of P . If at some point there doesn't exist a transition, then the pattern P doesn't appear as a substring of T . If we can process the entire string P this way, then the string appears in T .

It is clear that this will take $O(\text{length}(P))$ time for each string P . Moreover the algorithm actually finds the length of the longest prefix of P that appears in the text.

Number of different substrings

Given a string S . You want to compute the number of different substrings.

Let us build a suffix automaton for the string S .

Each substring of S corresponds to some path in the automaton. Therefore the number of different substrings is equal to the number of different paths in the automaton starting at t_0 .

Given that the suffix automaton is a directed acyclic graph, the number of different ways can be computed using dynamic programming.

Namely, let $d[v]$ be the number of ways, starting at the state v (including the path of length zero). Then we have the recursion:

$$d[v] = 1 + \sum_{w: (v, w, c) \in \text{DAG}} d[w]$$

i.e. $d[v]$ can be expressed as the sum of answers for all ends of the transitions of v .

The number of different substrings is the value $d[t_0] - 1$ (since we don't count the empty substring).

Total time complexity: $O(\text{length}(S))$

Alternatively, we can take advantage of the fact that each state v matches to substrings of length $[\text{minlen}(v), \text{len}(v)]$. Therefore, given $\text{minlen}(v) = 1 + \text{len}(\text{link}(v))$, we have total distinct substrings at state v being $\text{len}(v) - \text{minlen}(v) + 1 = \text{len}(v) - (1 + \text{len}(\text{link}(v))) + 1 = \text{len}(v) - \text{len}(\text{link}(v))$.

This is demonstrated succinctly below:

```
long long get_diff_strings(){
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        tot += st[i].len - st[st[i].link].len;
    }
    return tot;
}
```

While this is also $O(\text{length}(S))$, it requires no extra space and no recursive calls, consequently running faster in practice.

Total length of all different substrings

Given a string S . We want to compute the total length of all its various substrings.

The solution is similar to the previous one, only now it is necessary to consider two quantities for the dynamic programming part: the number of different substrings $d[v]$ and their total length $\text{ans}[v]$.

We already described how to compute $d[v]$ in the previous task. The value $\text{ans}[v]$ can be computed using the recursion:

$$\text{ans}[v] = \sum_{w: (v, w, c) \in \text{DAG}} d[w] + \text{ans}[w]$$

We take the answer of each adjacent vertex w , and add to it $d[w]$ (since every substring is one character longer when starting from the state v).

Again this task can be computed in $O(\text{length}(S))$ time.

Alternatively, we can, again, take advantage of the fact that each state v matches to substrings of length $[\min \text{len}(v), \text{len}(v)]$. Since $\min \text{len}(v) = 1 + \text{len}(\text{link}(v))$ and the arithmetic series formula $S_n = n \cdot \frac{a_1 + a_n}{2}$ (where S_n denotes the sum of n terms, a_1 representing the first term, and a_n representing the last), we can compute the length of substrings at a state in constant time. We then sum up these totals for each state $v \neq t_0$ in the automaton. This is shown by the code below:

```
long long get_tot_len_diff_substings() {
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        long long shortest = st[st[i].link].len + 1;
        long long longest = st[i].len;

        long long num_strings = longest - shortest + 1;
        long long cur = num_strings * (longest + shortest) / 2;
        tot += cur;
    }
    return tot;
}
```

This approach runs in $O(\text{length}(S))$ time, but experimentally runs 20x faster than the memoized dynamic programming version on randomized strings. It requires no extra space and no recursion.

Lexicographically k -th substring

Given a string S . We have to answer multiple queries. For each given number K_i we have to find the K_i -th string in the lexicographically ordered list of all substrings.

The solution to this problem is based on the idea of the previous two problems. The lexicographically k -th substring corresponds to the lexicographically k -th path in the suffix automaton. Therefore after counting the number of paths from each state, we can easily search for the k -th path starting from the root of the automaton.

This takes $O(\text{length}(S))$ time for preprocessing and then $O(\text{length}(\text{ans}) \cdot k)$ for each query (where ans is the answer for the query and k is the size of the alphabet).

Smallest cyclic shift

Given a string S . We want to find the lexicographically smallest cyclic shift.

We construct a suffix automaton for the string $S + S$. Then the automaton will contain in itself as paths all the cyclic shifts of the string S .

Consequently the problem is reduced to finding the lexicographically smallest path of length $\text{length}(S)$, which can be done in a trivial way: we start in the initial state and greedily pass through the transitions with the minimal character.

Total time complexity is $O(\text{length}(S))$.

Number of occurrences

For a given text T . We have to answer multiple queries. For each given pattern P we have to find out how many times the string P appears in the string T as a substring.

We construct the suffix automaton for the text T .

Next we do the following preprocessing: for each state v in the automaton we calculate the number $\text{cnt}[v]$ that is equal to the size of the set $\text{endpos}(v)$. In fact all strings corresponding to the same state v appear in the text T an equal amount of times, which is equal to the number of positions in the set endpos .

However we cannot construct the sets endpos explicitly, therefore we only consider their sizes cnt .

To compute them we proceed as follows. For each state, if it was not created by cloning (and if it is not the initial state t_0), we initialize it with $\text{cnt} = 1$. Then we will go through all states in decreasing order of their length len , and add the current value $\text{cnt}[v]$ to the suffix links:

$$\text{cnt}[\text{link}(v)] += \text{cnt}[v]$$

This gives the correct value for each state.

Why is this correct? The total number of states obtained *not* via cloning is exactly $\text{length}(T)$, and the first i of them appeared when we added the first i characters. Consequently for each of these states we count the corresponding position at which it was processed. Therefore initially we have $\text{cnt} = 1$ for each such state, and $\text{cnt} = 0$ for all other.

Then we apply the following operation for each v : $\text{cnt}[\text{link}(v)] += \text{cnt}[v]$. The meaning behind this is, that if a string v appears $\text{cnt}[v]$ times, then also all its suffixes appear at the exact same end positions, therefore also $\text{cnt}[v]$ times.

Why don't we overcount in this procedure (i.e. don't count some positions twice)? Because we add the positions of a state to only one other state, so it can not happen that one state directs its positions to another state twice in two different ways.

Thus we can compute the quantities cnt for all states in the automaton in $O(\text{length}(T))$ time.

After that answering a query by just looking up the value $\text{cnt}[t]$, where t is the state corresponding to the pattern, if such a state exists. Otherwise answer with 0. Answering a query takes $O(\text{length}(P))$ time.

First occurrence position

Given a text T and multiple queries. For each query string P we want to find the position of the first occurrence of P in the string T (the position of the beginning of P).

We again construct a suffix automaton. Additionally we precompute the position firstpos for all states in the automaton, i.e. for each state v we want to find the position $\text{firstpos}[v]$ of the end of the first occurrence. In other words, we want to find in advance the minimal element of each set endpos (since obviously cannot maintain all sets endpos explicitly).

To maintain these positions firstpos we extend the function `sa_extend()`. When we create a new state cur , we set:

$$\text{firstpos}(\text{cur}) = \text{len}(\text{cur}) - 1$$

And when we clone a vertex q as clone , we set:

$$\text{firstpos}(\text{clone}) = \text{firstpos}(q)$$

(since the only other option for a value would be $\text{firstpos}(\text{cur})$ which is definitely too big)

Thus the answer for a query is simply $\text{firstpos}(t) - \text{length}(P) + 1$, where t is the state corresponding to the string P . Answering a query again takes only $O(\text{length}(P))$ time.

All occurrence positions

This time we have to display all positions of the occurrences in the string T .

Again we construct a suffix automaton for the text T . Similar as in the previous task we compute the position firstpos for all states.

Clearly $\text{firstpos}(t)$ is part of the answer, if t is the state corresponding to a query string P . So we took into account the state of the automaton containing P . What other states do we need to take into account? All states that correspond to strings for which P is a suffix. In other words we need to find all the states that can reach the state t via suffix links.

Therefore to solve the problem we need to save for each state a list of suffix references leading to it. The answer to the query then will then contain all firstpos for each state that we can find on a DFS / BFS starting from the state t using only the suffix references.

Overall, this requires $O(\text{length}(T))$ for preprocessing and $O(\text{length}(P) + \text{answer}(P))$ for each request, where $\text{answer}(P)$ — this is the size of the answer.

First, we walk down the automaton for each character in the pattern to find our starting node requiring $O(\text{length}(P))$. Then, we use our workaround which will work in time $O(\text{answer}(P))$, because we will not visit a state twice (because only one suffix link leaves each state, so there cannot be two different paths leading to the same state).

We only must take into account that two different states can have the same `firstpos` value. This happens if one state was obtained by cloning another. However, this doesn't ruin the complexity, since each state can only have at most one clone.

Moreover, we can also get rid of the duplicate positions, if we don't output the positions from the cloned states. In fact a state, that a cloned state can reach, is also reachable from the original state. Thus if we remember the flag `is_cloned` for each state, we can simply ignore the cloned states and only output `firstpos` for all other states.

Here are some implementation sketches:

```
struct state {
    ...
    bool is_clone;
    int first_pos;
    vector<int> inv_link;
};

// after constructing the automaton
for (int v = 1; v < sz; v++) {
    st[st[v].link].inv_link.push_back(v);
}

// output all positions of occurrences
void output_all_occurrences(int v, int P_length) {
    if (!st[v].is_clone)
        cout << st[v].first_pos - P_length + 1 << endl;
    for (int u : st[v].inv_link)
        output_all_occurrences(u, P_length);
}
```

Shortest non-appearing string

Given a string S and a certain alphabet. We have to find a string of the smallest length, that doesn't appear in S .

We will apply dynamic programming on the suffix automaton built for the string S .

Let $d[v]$ be the answer for the node v , i.e. we already processed part of the substring, are currently in the state v , and want to find the smallest number of characters that have to be added to find a non-existent transition. Computing $d[v]$ is very simple. If there is not transition using at least one character of the alphabet, then $d[v] = 1$. Otherwise one character is not enough, and so we need to take the minimum of all answers of all transitions:

$$d[v] = 1 + \min_{w: (v, w, c) \in SA} d[w].$$

The answer to the problem will be $d[t_0]$, and the actual string can be restored using the computed array $d[]$.

Longest common substring of two strings

Given two strings S and T . We have to find the longest common substring, i.e. such a string X that appears as a substring in S and also in T .

We construct a suffix automaton for the string S .

We will now take the string T , and for each prefix look for the longest suffix of this prefix in S . In other words, for each position in the string T , we want to find the longest common substring of S and T ending in that position.

For this we will use two variables, the **current state** v , and the **current length** l . These two variables will describe the current matching part: its length and the state that corresponds to it.

Initially $v = t_0$ and $l = 0$, i.e. the match is empty.

Now let us describe how we can add a character $T[i]$ and recalculate the answer for it.

- If there is a transition from v with the character $T[i]$, then we simply follow the transition and increase l by one.
- If there is no such transition, we have to shorten the current matching part, which means that we need to follow the suffix link: $v = \text{link}(v)$. At the same time, the current length has to be shortened. Obviously we need to assign $l = \text{len}(v)$, since after passing through the suffix link we end up in state whose corresponding longest string is a substring.
- If there is still no transition using the required character, we repeat and again go through the suffix link and decrease l , until we find a transition or we reach the fictional state -1 (which means that the symbol $T[i]$ doesn't appear at all in S , so we assign $v = l = 0$).

The answer to the task will be the maximum of all the values l .

The complexity of this part is $O(\text{length}(T))$, since in one move we can either increase l by one, or make several passes through the suffix links, each one ends up reducing the value l .

Implementation:

```
string lcs (string S, string T) {
    sa_init();
    for (int i = 0; i < S.size(); i++)
        sa_extend(S[i]);

    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < T.size(); i++) {
        while (v && !st[v].next.count(T[i])) {
            v = st[v].link;
            l = st[v].len;
        }
        if (st[v].next.count(T[i])) {
            v = st[v].next[T[i]];
            l++;
        }
        if (l > best) {
            best = l;
            bestpos = i;
        }
    }
    return T.substr(bestpos - best + 1, best);
}
```

Largest common substring of multiple strings

There are k strings S_i given. We have to find the longest common substring, i.e. such a string X that appears as substring in each string S_i .

We join all strings into one large string T , separating the strings by a special characters D_i (one for each string):

$$T = S_1 + D_1 + S_2 + D_2 + \dots + S_k + D_k.$$

Then we construct the suffix automaton for the string T .

Now we need to find a string in the machine, which is contained in all the strings S_i , and this can be done by using the special added characters.

Note that if a substring is included in some string S_j , then in the suffix automaton exists a path starting from this substring containing the character D_j and not containing the other characters $D_1, \dots, D_{j-1}, D_{j+1}, \dots, D_k$.

Thus we need to calculate the attainability, which tells us for each state of the machine and each symbol D_i if there exists such a path. This can easily be computed by DFS or BFS and dynamic programming. After that, the answer to the problem will be the string $\text{longest}(v)$ for the state v , from which the paths were exists for all special characters.

Practice Problems

- [CSES - Finding Patterns](#)
- [CSES - Counting Patterns](#)
- [CSES - String Matching](#)
- [CSES - Patterns Positions](#)
- [CSES - Distinct Substrings](#)

- [CSES - Word Combinations](#)
- [CSES - String Distribution](#)
- [AtCoder - K-th Substring](#)
- [SPOJ - SUBLEX](#)
- [Codeforces - Cyclical Quest](#)
- [Codeforces - String](#)

Contributors:

[Jakob Kogler](#) (89.12%) [Michael Hayter](#) (4.81%) [Atharva Thorve](#) (1.9%) [Oleksandr Kulkov](#) (0.89%) [iagorrr](#) (0.89%) [Tarun Khullar](#) (0.76%)
[James Wilson](#) (0.63%) [Ir1dXD](#) (0.25%) [Alfredo Beaumont](#) (0.13%) [Bergur Snorrason](#) (0.13%) [Le Bao Hiep](#) (0.13%) [Md. Abdulla Al Mamun](#) (0.13%)
[Robin Hafid Quintero Lopez](#) (0.13%) [wiktorkuchta](#) (0.13%)

Translated

From: e-maxx.ru

Lyndon factorization

Lyndon factorization

First let us define the notion of the Lyndon factorization.

A string is called **simple** (or a Lyndon word), if it is strictly **smaller than** any of its own nontrivial **suffixes**. Examples of simple strings are: a, b, ab, aab, abb, ababb, abcd. It can be shown that a string is simple, if and only if it is strictly **smaller than** all its nontrivial **cyclic shifts**.

Next, let there be a given string s . The **Lyndon factorization** of the string s is a factorization $s = w_1 w_2 \dots w_k$, where all strings w_i are simple, and they are in non-increasing order $w_1 \geq w_2 \geq \dots \geq w_k$.

It can be shown, that for any string such a factorization exists and that it is unique.

Duval algorithm

The Duval algorithm constructs the Lyndon factorization in $O(n)$ time using $O(1)$ additional memory.

First let us introduce another notion: a string t is called **pre-simple**, if it has the form $t = ww \dots w\bar{w}$, where w is a simple string and $\bar{w}$ is a prefix of w (possibly empty). A simple string is also pre-simple.

The Duval algorithm is greedy. At any point during its execution, the string s will actually be divided into three strings $s = s_1 s_2 s_3$, where the Lyndon factorization for s_1 is already found and finalized, the string s_2 is pre-simple (and we know the length of the simple string in it), and s_3 is completely untouched. In each iteration the Duval algorithm takes the first character of the string s_3 and tries to append it to the string s_2 . If s_2 is no longer pre-simple, then the Lyndon factorization for some part of s_2 becomes known, and this part goes to s_1 .

Let's describe the algorithm in more detail. The pointer i will always point to the beginning of the string s_2 . The outer loop will be executed as long as $i < n$. Inside the loop we use two additional pointers, j which points to the beginning of s_3 , and k which points to the current character that we are currently comparing to. We want to add the character $s[j]$ to the string s_2 , which requires a comparison with the character $s[k]$. There can be three different cases:

- $s[j] = s[k]$: if this is the case, then adding the symbol $s[j]$ to s_2 doesn't violate its pre-simplicity. So we simply increment the pointers j and k .
- $s[j] > s[k]$: here, the string $s_2 + s[j]$ becomes simple. We can increment j and reset k back to the beginning of s_2 , so that the next character can be compared with the beginning of the simple word.
- $s[j] < s[k]$: the string $s_2 + s[j]$ is no longer pre-simple. Therefore we will split the pre-simple string s_2 into its simple strings and the remainder, possibly empty. The simple string will have the length $j - k$. In the next iteration we start again with the remaining s_2 .

Implementation

Here we present the implementation of the Duval algorithm, which will return the desired Lyndon factorization of a given string s .

```
vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        factorization.push_back(s.substr(i, j - i));
        i = j;
    }
    return factorization;
}
```

```

    }
    while (i <= k) {
        factorization.push_back(s.substr(i, j - k));
        i += j - k;
    }
}
return factorization;
}

```

Complexity

Let us estimate the running time of this algorithm.

The **outer while loop** does not exceed n iterations, since at the end of each iteration i increases. Also the second inner while loop runs in $O(n)$, since it only outputs the final factorization.

So we are only interested in the **first inner while loop**. How many iterations does it perform in the worst case? It's easy to see that the simple words that we identify in each iteration of the outer loop are longer than the remainder that we additionally compared. Therefore also the sum of the remainders will be smaller than n , which means that we only perform at most $O(n)$ iterations of the first inner while loop. In fact the total number of character comparisons will not exceed $4n - 3$.

Finding the smallest cyclic shift

Let there be a string s . We construct the Lyndon factorization for the string $s + s$ (in $O(n)$ time). We will look for a simple string in the factorization, which starts at a position less than n (i.e. it starts in the first instance of s), and ends in a position greater than or equal to n (i.e. in the second instance) of s). It is stated, that the position of the start of this simple string will be the beginning of the desired smallest cyclic shift. This can be easily verified using the definition of the Lyndon decomposition.

The beginning of the simple block can be found easily - just remember the pointer i at the beginning of each iteration of the outer loop, which indicated the beginning of the current pre-simple string.

So we get the following implementation:

```

string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k)
            i += j - k;
    }
    return s.substr(ans, n / 2);
}

```

Problems

- [UVA #719 - Glass Beads](#)

Contributors:

[Jakob Kogler](#) (94.49%) [Oleksandr Kulkov](#) (4.72%) [wiktorkuchta](#) (0.79%)

Translated

From: e-maxx.ru

Expression parsing

A string containing a mathematical expression containing numbers and various operators is given. We have to compute the value of it in $O(n)$, where n is the length of the string.

The algorithm discussed here translates an expression into the so-called **reverse Polish notation** (explicitly or implicitly), and evaluates this expression.

Reverse Polish notation

The reverse Polish notation is a form of writing mathematical expressions, in which the operators are located after their operands. For example the following expression

$$a + b * c * d + (e - f) * (g * h + i)$$

can be written in reverse Polish notation in the following way:

$$abc * d * + ef - gh * i + *$$

The reverse Polish notation was developed by the Australian philosopher and computer science specialist Charles Hamblin in the mid 1950s on the basis of the Polish notation, which was proposed in 1920 by the Polish mathematician Jan Łukasiewicz.

The convenience of the reverse Polish notation is, that expressions in this form are very **easy to evaluate** in linear time. We use a stack, which is initially empty. We will iterate over the operands and operators of the expression in reverse Polish notation. If the current element is a number, then we put the value on top of the stack, if the current element is an operator, then we get the top two elements from the stack, perform the operation, and put the result back on top of the stack. In the end there will be exactly one element left in the stack, which will be the value of the expression.

Obviously this simple evaluation runs in $O(n)$ time.

Parsing of simple expressions

For the time being we only consider a simplified problem: we assume that all operators are **binary** (i.e. they take two arguments), and all are **left-associative** (if the priorities are equal, they get executed from left to right). Parentheses are allowed.

We will set up two stacks: one for numbers, and one for operators and parentheses. Initially both stacks are empty. For the second stack we will maintain the condition that all operations are ordered by strict descending priority. If there are parentheses on the stack, then each block of operators (corresponding to one pair of parenthesis) is ordered, and the entire stack is not necessarily ordered.

We will iterate over the characters of the expression from left to right. If the current character is a digit, then we put the value of this number on the stack. If the current character is an opening parenthesis, then we put it on the stack. If the current character is a closing parenthesis, then we execute all operators on the stack until we reach the opening bracket (in other words we perform all operations inside the parenthesis). Finally if the current character is an operator, then while the top of the stack has an operator with the same or higher priority, we will execute this operation, and put the new operation on the stack.

After we processed the entire string, some operators might still be in the stack, so we execute them.

Here is the implementation of this method for the four operators $+$ $-$ $*$ $/$:

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}
```

```

int priority (char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    int r = st.top(); st.pop();
    int l = st.top(); st.pop();
    switch (op) {
        case '+': st.push(l + r); break;
        case '-': st.push(l - r); break;
        case '*': st.push(l * r); break;
        case '/': st.push(l / r); break;
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i]))
            continue;

        if (s[i] == '(') {
            op.push('(');
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            while (!op.empty() && priority(op.top()) >= priority(cur_op)) {
                process_op(st, op.top());
                op.pop();
            }
            op.push(cur_op);
        } else {
            int number = 0;
            while (i < (int)s.size() && isalnum(s[i]))
                number = number * 10 + s[i++] - '0';
            --i;
            st.push(number);
        }
    }

    while (!op.empty()) {
        process_op(st, op.top());
        op.pop();
    }
    return st.top();
}

```

Thus we learned how to calculate the value of an expression in $O(n)$, at the same time we implicitly used the reverse Polish notation. By slightly modifying the above implementation it is also possible to obtain the expression in reverse Polish notation in an explicit form.

Unary operators

Now suppose that the expression also contains **unary** operators (operators that take one argument). The unary plus and unary minus are common examples of such operators.

One of the differences in this case, is that we need to determine whether the current operator is a unary or a binary one.

You can notice, that before an unary operator, there always is another operator or an opening parenthesis, or nothing at all (if it is at the very beginning of the expression). On the contrary before a binary operator there will always be an operand (number) or a closing parenthesis. Thus it is easy to flag whether the next operator can be unary or not.

Additionally we need to execute a unary and a binary operator differently. And we need to chose the priority of a unary operator higher than all of the binary operators.

In addition it should be noted, that some unary operators (e.g. unary plus and unary minus) are actually **right-associative**.

Right-associativity

Right-associative means, that whenever the priorities are equal, the operators must be evaluated from right to left.

As noted above, unary operators are usually right-associative. Another example for an right-associative operator is the exponentiation operator ($a \wedge b \wedge c$ is usually perceived as a^{b^c} and not as $(a^b)^c$).

What difference do we need to make in order to correctly handle right-associative operators? It turns out that the changes are very minimal. The only difference will be, if the priorities are equal we will postpone the execution of the right-associative operation.

The only line that needs to be replaced is

```
while (!op.empty() && priority(op.top()) >= priority(cur_op))
```

with

```
while (!op.empty() && (
    (left_assoc(cur_op) && priority(op.top()) >= priority(cur_op)) ||
    (!left_assoc(cur_op) && priority(op.top()) > priority(cur_op))
))
```

where `left_assoc` is a function that decides if an operator is left-associative or not.

Here is an implementation for the binary operators `+` `-` `*` `/` and the unary operators `+` and `-`.

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

bool is_unary(char c) {
    return c == '+' || c == '-';
}

int priority(char op) {
    if (op < 0) // unary operator
        return 3;
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    if (op < 0) {
        int l = st.top(); st.pop();
        switch (-op) {
            case '+': st.push(l); break;
            case '-': st.push(-l); break;
        }
    } else {
        int r = st.top(); st.pop();
        int l = st.top(); st.pop();
        switch (op) {
            case '+': st.push(l + r); break;
            case '-': st.push(l - r); break;
            case '*': st.push(l * r); break;
            case '/': st.push(l / r); break;
        }
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < (int)s.size(); i++) {
```

```

if (delim(s[i]))
    continue;

if (s[i] == '(') {
    op.push('(');
    may_be_unary = true;
} else if (s[i] == ')') {
    while (op.top() != '(') {
        process_op(st, op.top());
        op.pop();
    }
    op.pop();
    may_be_unary = false;
} else if (is_op(s[i])) {
    char cur_op = s[i];
    if (may_be_unary && is_unary(cur_op))
        cur_op = -cur_op;
    while (!op.empty() && (
        (cur_op >= 0 && priority(op.top()) >= priority(cur_op)) ||
        (cur_op < 0 && priority(op.top()) > priority(cur_op))
    )) {
        process_op(st, op.top());
        op.pop();
    }
    op.push(cur_op);
    may_be_unary = true;
} else {
    int number = 0;
    while (i < (int)s.size() && isalnum(s[i]))
        number = number * 10 + s[i++] - '0';
    --i;
    st.push(number);
    may_be_unary = false;
}
}

while (!op.empty()) {
    process_op(st, op.top());
    op.pop();
}

return st.top();
}

```

Contributors:

[Jakob Kogler](#) (96.9%)
 [Oleksandr Kulkov](#) (2.33%)
 [Piyush Naithani](#) (0.39%)
 [wiktorkuchta](#) (0.39%)

Translated

From: e-maxx.ru

Manacher's Algorithm - Finding all sub-palindromes in $O(N)$

Statement

Given string s with length n . Find all the pairs (i, j) such that substring $s[i \dots j]$ is a palindrome. String t is a palindrome when $t = t_{\text{rev}}$ (t_{rev} is a reversed string for t).

More precise statement

In the worst case string might have up to $O(n^2)$ palindromic substrings, and at the first glance it seems that there is no linear algorithm for this problem.

But the information about the palindromes can be kept in a **compact way**: for each position i we will find the number of non-empty palindromes centered at this position.

Palindromes with a common center form a contiguous chain, that is if we have a palindrome of length l centered in i , we also have palindromes of lengths $l - 2$, $l - 4$ and so on also centered in i . Therefore, we will collect the information about all palindromic substrings in this way.

Palindromes of odd and even lengths are accounted for separately as $d_{\text{odd}}[i]$ and $d_{\text{even}}[i]$. For the palindromes of even length we assume that they're centered in the position i if their two central characters are $s[i]$ and $s[i - 1]$.

For instance, string $s = \text{ababab}$ has three palindromes with odd length with centers in the position $s[3] = \text{b}$, i. e. $d_{\text{odd}}[3] = 3$:

$d_{\text{odd}}[3] = 3$
 $\square \square \square \square \square$
 $\text{a b a } \text{b} \text{ a b c}$
 $\square \square \square \square$
 s_3

And string $s = \text{cbabdb}$ has two palindromes with even length with centers in the position $s[3] = \text{a}$, i. e. $d_{\text{even}}[3] = 2$:

$d_{\text{even}}[3] = 2$
 $\square \square \square \square$
 $\text{c b a } \text{a} \text{ b d}$
 $\square \square \square \square$
 s_3

It's a surprising fact that there is an algorithm, which is simple enough, that calculates these "palindromity arrays" $d_{\text{odd}}[]$ and $d_{\text{even}}[]$ in linear time. The algorithm is described in this article.

Solution

In general, this problem has many solutions: with [String Hashing](#) it can be solved in $O(n \cdot \log n)$, and with [Suffix Trees](#) and fast LCA this problem can be solved in $O(n)$.

But the method described here is **sufficiently** simpler and has less hidden constant in time and memory complexity. This algorithm was discovered by **Glenn K. Manacher** in 1975.

Another modern way to solve this problem and to deal with palindromes in general is through the so-called palindromic tree, or eertree.

Trivial algorithm

To avoid ambiguities in the further description we denote what "trivial algorithm" is.

It's the algorithm that does the following. For each center position i it tries to increase the answer by one as long as it's possible, comparing a pair of corresponding characters each time.

Such an algorithm is slow, it can calculate the answer only in $O(n^2)$.

The implementation of the trivial algorithm is:

```
vector<int> manacher_odd_trivial(string s) {
    int n = s.size();
    s = "$" + s + "A";
    vector<int> p(n + 2);
    for(int i = 1; i <= n; i++) {
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

Terminal characters `$` and `A` were used to avoid dealing with ends of the string separately.

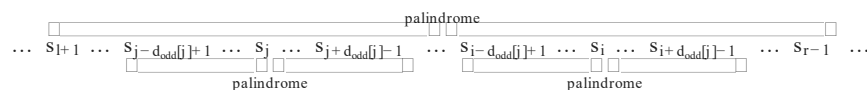
Manacher's algorithm

We describe the algorithm to find all the sub-palindromes with odd length, i. e. to calculate $d_{\text{odd}}[]$.

For fast calculation we'll maintain the **exclusive borders** (l, r) of the rightmost found (sub-)palindrome (i. e. the current rightmost (sub-)palindrome is $s[l + 1]s[l + 2] \dots s[r - 1]$). Initially we set $l = 0, r = 1$, which corresponds to the empty string.

So, we want to calculate $d_{\text{odd}}[i]$ for the next i , and all the previous values in $d_{\text{odd}}[]$ have been already calculated. We do the following:

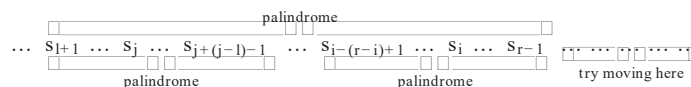
- If j is outside the current sub-palindrome, i. e. $i \geq r$, we'll just launch the trivial algorithm.
So we'll increase $d_{\text{odd}}[i]$ consecutively and check each time if the current rightmost substring $[i - d_{\text{odd}}[i] \dots i + d_{\text{odd}}[i]]$ is a palindrome. When we find the first mismatch or meet the boundaries of s , we'll stop. In this case we've finally calculated $d_{\text{odd}}[i]$. After this, we must not forget to update (l, r) . r should be updated in such a way that it represents the last index of the current rightmost sub-palindrome.
- Now consider the case when $i \leq r$. We'll try to extract some information from the already calculated values in $d_{\text{odd}}[]$. So, let's find the "mirror" position of i in the sub-palindrome (l, r) , i. e. we'll get the position $j = l + (r - i)$, and we check the value of $d_{\text{odd}}[j]$. Because j is the position symmetrical to i with respect to $(l + r) / 2$, we can **almost always** assign $d_{\text{odd}}[i] = d_{\text{odd}}[j]$. Illustration of this (palindrome around j is actually "copied" into the palindrome around i):



But there is a **tricky case** to be handled correctly: when the "inner" palindrome reaches the borders of the "outer" one, i. e. $j - d_{\text{odd}}[j] \leq l$ (or, which is the same, $i + d_{\text{odd}}[j] \geq r$). Because the symmetry outside the "outer" palindrome is not guaranteed, just assigning $d_{\text{odd}}[i] = d_{\text{odd}}[j]$ will be incorrect: we do not have enough data to state that the palindrome in the position i has the same length.

Actually, we should restrict the length of our palindrome for now, i. e. assign $d_{\text{odd}}[i] = r - i$, to handle such situations correctly. After this we'll run the trivial algorithm which will try to increase $d_{\text{odd}}[i]$ while it's possible.

Illustration of this case (the palindrome with center j is restricted to fit the "outer" palindrome):



It is shown in the illustration that though the palindrome with center j could be larger and go outside the "outer" palindrome, but with i as the center we can use only the part that entirely fits into the "outer" palindrome. But the answer for the position i ($d_{\text{odd}}[i]$) can be much bigger than this part, so next we'll run our trivial algorithm that will try to grow it outside our "outer" palindrome, i. e. to the region "try moving here".

Again, we should not forget to update the values (l, r) after calculating each $d_{\text{odd}}[i]$.

Complexity of Manacher's algorithm

At the first glance it's not obvious that this algorithm has linear time complexity, because we often run the naive algorithm while searching the answer for a particular position.

However, a more careful analysis shows that the algorithm is linear. In fact, [Z-function building algorithm](#), which looks similar to this algorithm, also works in linear time.

We can notice that every iteration of trivial algorithm increases r by one. Also r cannot be decreased during the algorithm. So, trivial algorithm will make $O(n)$ iterations in total.

Other parts of Manacher's algorithm work obviously in linear time. Thus, we get $O(n)$ time complexity.

Implementation of Manacher's algorithm

For calculating $d_{\text{odd}}[]$, we get the following code. Things to note:

- i is the index of the center letter of the current palindrome.
- If i exceeds r , $d_{\text{odd}}[i]$ is initialized to 0.
- If i does not exceed r , $d_{\text{odd}}[i]$ is either initialized to the $d_{\text{odd}}[j]$, where j is the mirror position of i in (l, r) , or $d_{\text{odd}}[i]$ is restricted to the size of the "outer" palindrome.
- The while loop denotes the trivial algorithm. We launch it irrespective of the value of k .
- If the size of palindrome centered at i is x , then $d_{\text{odd}}[i]$ stores $\frac{x+1}{2}$.

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "A";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = min(r - i, p[l + (r - i)]);
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

Working with parities

Although it is possible to implement Manacher's algorithm for odd and even lengths separately, the implementation of the version for even lengths is often deemed more difficult, as it is less natural and easily leads to off-by-one errors.

To mitigate this, it is possible to reduce the whole problem to the case when we only deal with the palindromes of odd length. To do this, we can put an additional `#` character between each letter in the string and also in the beginning and the end of the string:

$$abcbcb\text{a} \rightarrow \# \text{a}\# \text{b}\# \text{c}\# \text{b}\# \text{c}\# \text{b}\# \text{a}\# ,$$

$$d = [1, 2, 1, 2, 1, 4, 1, 8, 1, 4, 1, 2, 1, 2, 1].$$

As you can see, $d[2i] = 2d_{\text{even}}[i] + 1$ and $d[2i + 1] = 2d_{\text{odd}}[i]$ where d denotes the Manacher array for odd-length palindromes in `#`-joined string, while d_{odd} and d_{even} correspond to the arrays defined above in the initial string.

Indeed, `#` characters do not affect the odd-length palindromes, which are still centered in the initial string's characters, but now even-length palindromes of the initial string are odd-length palindromes of the new string centered in `#` characters.

Note that $d[2i]$ and $d[2i + 1]$ are essentially the increased by 1 lengths of the largest odd- and even-length palindromes centered in i correspondingly.

The reduction is implemented in the following way:

```
vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}
```

For simplicity, splitting the array into d_{odd} and d_{even} as well as their explicit calculation is omitted.

Problems

- [Library Checker - Enumerate Palindromes](#)
- [Longest Palindrome](#)
- [UVA 11475 - Extend to Palindrome](#)
- [GYM - \(Q\) QueryreuQ](#)
- [CF - Prefix-Suffix Palindrome](#)
- [SPOJ - Number of Palindromes](#)
- [Kattis - Palindromes](#)

Contributors:

[Alexander Kernozhitsky](#) (45.05%) [Oleksandr Kulkov](#) (27.73%) [bruised_samurai](#) (11.39%) [Jakob Kogler](#) (5.45%) [Sunit Gautam](#) (3.47%) [Yurii A.](#) (1.98%)
[Dip Turkar](#) (0.99%) [Felipe Noronha](#) (0.99%) [sanct](#) (0.99%) [Anton Ålgmyr](#) (0.5%) [Tarun Khullar](#) (0.5%) [Yan Huang](#) (0.5%) [navneet-iitbhu](#) (0.5%)

Translated

From: e-maxx.ru

Finding repetitions

Given a string s of length n .

A **repetition** is two occurrences of a string in a row. In other words a repetition can be described by a pair of indices $i < j$ such that the substring $s[i \dots j]$ consists of two identical strings written after each other.

The challenge is to **find all repetitions** in a given string s . Or a simplified task: find **any** repetition or find the **longest** repetition.

The algorithm described here was published in 1982 by Main and Lorentz.

Example

Consider the repetitions in the following example string:

acababace

The string contains the following three repetitions:

- $s[2 \dots 5] = abab$
- $s[3 \dots 6] = baba$
- $s[7 \dots 8] = ee$

Another example:

abaaba

Here there are only two repetitions

- $s[0 \dots 5] = abaaba$
- $s[2 \dots 3] = aa$

Number of repetitions

In general there can be up to $O(n^2)$ repetitions in a string of length n . An obvious example is a string consisting of n times the same letter, in this case any substring of even length is a repetition. In general any periodic string with a short period will contain a lot of repetitions.

On the other hand this fact does not prevent computing the number of repetitions in $O(n \log n)$ time, because the algorithm can give the repetitions in compressed form, in groups of several pieces at once.

There is even the concept, that describes groups of periodic substrings with tuples of size four. It has been proven that the number of such groups is at most linear with respect to the string length.

Also, here are some more interesting results related to the number of repetitions:

- The number of primitive repetitions (those whose halves are not repetitions) is at most $O(n \log n)$.
- If we encode repetitions with tuples of numbers (called Crochemore triples) (i, p, r) (where i is the position of the beginning, p the length of the repeating substring, and r the number of repetitions), then all repetitions can be described with $O(n \log n)$ such triples.
- Fibonacci strings, defined as

$$t_0 = a,$$

$$t_1 = b,$$

$$t_i = t_{i-1} + t_{i-2},$$

are "strongly" periodic. The number of repetitions in the Fibonacci string f_i , even in the compressed with Crochemore triples, is $O(f_n \log f_n)$. The number of primitive repetitions is also $O(f_n \log f_n)$.

Main-Lorentz algorithm

The idea behind the Main-Lorentz algorithm is **divide-and-conquer**.

It splits the initial string into halves, and computes the number of repetitions that lie completely in each half by two recursive calls. Then comes the difficult part. The algorithm finds all repetitions starting in the first half and ending in the second half (which we will call **crossing repetitions**). This is the essential part of the Main-Lorentz algorithm, and we will discuss it in detail here.

The complexity of divide-and-conquer algorithms is well researched. The [master theorem](#) says, that we will end up with an $O(n \log n)$ algorithm, if we can compute the crossing repetitions in $O(n)$ time.

Search for crossing repetitions

So we want to find all such repetitions that start in the first half of the string, let's call it u , and end in the second half, let's call it v :

$$s = u + v$$

Their lengths are approximately equal to the length of s divided by two.

Consider an arbitrary repetition and look at the middle character (more precisely the first character of the second half of the repetition). I.e. if the repetition is a substring $s[i \dots j]$, then the middle character is $(i + j + 1) / 2$.

We call a repetition **left** or **right** depending on which string this character is located - in the string u or in the string v . In other words a string is called left, if the majority of it lies in u , otherwise we call it right.

We will now discuss how to find **all left repetitions**. Finding all right repetitions can be done in the same way.

Let us denote the length of the left repetition by $2l$ (i.e. each half of the repetition has length l). Consider the first character of the repetition falling into the string v (it is at position $|u|$ in the string s). It coincides with the character l positions before it, let's denote this position $cntr$.

We will fixate this position $cntr$, and **look for all repetitions at this position** $cntr$.

For example:

$$\begin{array}{ccccccc} c & a & c & | & a & d & a \\ & & cntr & & & & \end{array}$$

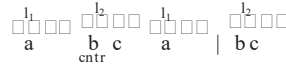
The vertical lines divides the two halves. Here we fixated the position $cntr = 1$, and at this position we find the repetition $caca$.

It is clear, that if we fixate the position $cntr$, we simultaneously fixate the length of the possible repetitions: $l = |u| - cntr$. Once we know how to find these repetitions, we will iterate over all possible values for $cntr$ from 0 to $|u| - 1$, and find all left crossover repetitions of length $l = |u|, |u| - 1, \dots, 1$.

Criterion for left crossing repetitions

Now, how can we find all such repetitions for a fixated $cntr$? Keep in mind that there still can be multiple such repetitions.

Let's again look at a visualization, this time for the repetition $abcaabc$:



Here we denoted the lengths of the two pieces of the repetition with l_1 and l_2 : l_1 is the length of the repetition up to the position $\text{cntr} - 1$, and l_2 is the length of the repetition from cntr to the end of the half of the repetition. We have $2l = l_1 + l_2 + l_1 + l_2$ as the total length of the repetition.

Let us generate **necessary and sufficient** conditions for such a repetition at position cntr of length $2l = 2(l_1 + l_2) = 2(|u| - \text{cntr})$:

- Let k_1 be the largest number such that the first k_1 characters before the position cntr coincide with the last k_1 characters in the string u :

$$u[\text{cntr} - k_1 \dots \text{cntr} - 1] = u[|u| - k_1 \dots |u| - 1]$$

- Let k_2 be the largest number such that the k_2 characters starting at position cntr coincide with the first k_2 characters in the string v :

$$u[\text{cntr} \dots \text{cntr} + k_2 - 1] = v[0 \dots k_2 - 1]$$

- Then we have a repetition exactly for any pair (l_1, l_2) with

$$l_1 \leq k_1,$$

$$l_2 \leq k_2.$$

To summarize:

- We fixate a specific position cntr .
- All repetition which we will find now have length $2l = 2(|u| - \text{cntr})$. There might be multiple such repetitions, they depend on the lengths l_1 and $l_2 = l - l_1$.
- We find k_1 and k_2 as described above.
- Then all suitable repetitions are the ones for which the lengths of the pieces l_1 and l_2 satisfy the conditions:

$$l_1 + l_2 = l = |u| - \text{cntr}$$

$$l_1 \leq k_1,$$

$$l_2 \leq k_2.$$

Therefore the only remaining part is how we can compute the values k_1 and k_2 quickly for every position cntr . Luckily we can compute them in $O(1)$ using the [Z-function](#):

- To can find the value k_1 for each position by calculating the Z-function for the string $\bar{u}$ (i.e. the reversed string u). Then the value k_1 for a particular cntr will be equal to the corresponding value of the array of the Z-function.
- To precompute all values k_2 , we calculate the Z-function for the string $v + \# + u$ (i.e. the string u concatenated with the separator character $\#$ and the string v). Again we just need to look up the corresponding value in the Z-function to get the k_2 value.

So this is enough to find all left crossing repetitions.

Right crossing repetitions

For computing the right crossing repetitions we act similarly: we define the center cntr as the character corresponding to the last character in the string u .

Then the length k_1 will be defined as the largest number of characters before the position cntr (inclusive) that coincide with the last characters of the string u . And the length k_2 will be defined as the largest number of characters starting at $\text{cntr} + 1$ that coincide with the characters of the string v .

Thus we can find the values k_1 and k_2 by computing the Z-function for the strings $\overline{u} + \# + \overline{v}$ and v .

After that we can find the repetitions by looking at all positions cntr , and use the same criterion as we had for left crossing repetitions.

Implementation

The implementation of the Main-Lorentz algorithm finds all repetitions in form of peculiar tuples of size four: $(\text{cntr}, l, k_1, k_2)$ in $O(n \log n)$ time. If you only want to find the number of repetitions in a string, or only want to find the longest repetition in a string, this information is enough and the runtime will still be $O(n \log n)$.

Notice that if you want to expand these tuples to get the starting and end position of each repetition, then the runtime will be the runtime will be $O(n^2)$ (remember that there can be $O(n^2)$ repetitions). In this implementation we will do so, and store all found repetition in a vector of pairs of start and end indices.

```
vector<int> z_function(string const& s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

int get_z(vector<int> const& z, int i) {
    if (0 <= i && i < (int)z.size())
        return z[i];
    else
        return 0;
}

vector<pair<int, int>> repetitions;

void convert_to_repetitions(int shift, bool left, int cntr, int l, int k1, int k2) {
    for (int l1 = max(1, l - k2); l1 <= min(l, k1); l1++) {
        if (left && l1 == 1) break;
        int l2 = l - l1;
        int pos = shift + (left ? cntr - l1 : cntr - l - l1 + 1);
        repetitions.emplace_back(pos, pos + 2 * l1 - 1);
    }
}

void find_repetitions(string s, int shift = 0) {
    int n = s.size();
    if (n == 1)
        return;

    int nu = n / 2;
    int nv = n - nu;
    string u = s.substr(0, nu);
    string v = s.substr(nu);
    string ru(u.rbegin(), u.rend());
    string rv(v.rbegin(), v.rend());

    find_repetitions(u, shift);
    find_repetitions(v, shift + nu);

    vector<int> z1 = z_function(ru);
    vector<int> z2 = z_function(v + '#' + u);
    vector<int> z3 = z_function(ru + '#' + rv);
    vector<int> z4 = z_function(v);

    for (int cntr = 0; cntr < n; cntr++) {
        int l, k1, k2;
```

```
if (cntr < nu) {  
    l = nu - cntr;  
    k1 = get_z(z1, nu - cntr);  
    k2 = get_z(z2, nv + 1 + cntr);  
} else {  
    l = cntr - nu + 1;  
    k1 = get_z(z3, nu + 1 + nv - 1 - (cntr - nu));  
    k2 = get_z(z4, (cntr - nu) + 1);  
}  
if (k1 + k2 >= 1)  
    convert_to_repetitions(shift, cntr < nu, cntr, l, k1, k2);  
}  
}
```

Contributors:

[Jakob Kogler](#) (91.29%) [Oleksandr Kulkov](#) (7.58%) [Deji Oyerinde](#) (0.38%) [Michael Hayter](#) (0.38%) [wiktorkuchta](#) (0.38%)